PART 1 — Introduction and Topic 1: MIPS Addressing Modes

Scene 0.1 — Welcome and Roadmap

Visual directions: Title card: "UNIT III — PROCESSOR AND CONTROL UNIT". Below it, the syllabus line from the notes scrolls in: *Eight ideas – Functional Units – Basic Operational Concepts – Performance – Instructions: Language of the Computer – Operations, Operands – Instruction representation – MIPS addressing modes – A Basic MIPS implementation – Building a Datapath.*

Diagram instructions: A vertical roadmap with 10 numbered boxes. Boxes 1–3 (Addressing Modes, MIPS Implementation, Hardware Structure) are highlighted in gold with a "PRIORITY" badge. Boxes 4–10 are grey and light up later. A final box "Final Revision" sits at the bottom.

Narration:

> Hello, and welcome to Unit 3 — Processor and Control Unit. In this unit we open up the processor and see how it actually works: how an instruction is written, how it is stored as bits, how it finds its operands, and how the hardware inside the processor executes it. We will study this unit in a special order. First, three priority topics: MIPS addressing modes, the basic MIPS implementation, and the hardware structure of the MIPS processor. After those, we cover every remaining topic in the syllabus: the eight great ideas, the functional units, basic operational concepts, CPU performance, MIPS operations and operands, instruction formats, and building a datapath. We will finish with a complete revision section and an exam checklist. Because the priority topics come first, I will quickly give you the minimum background you need — just enough to understand addressing modes. Every one of these background ideas is taught again in full detail later in the video.

On-screen revision points:

- Unit 3 = Processor and Control Unit.
- Priority order: Addressing Modes → MIPS Implementation → Hardware Structure.

Scene 0.2 — Minimum Background: Instructions, Registers, Memory

Visual directions: Three icons appear one at a time: a "32 registers" cabinet, a long memory strip with addresses 0, 4, 8, 12 …, and a 32-bit instruction bar.

Diagram instructions:

- Register cabinet: 32 drawers labelled `$0 ($zero)`, `$8–$15 ($t0–$t7)`, `$16–$23 ($s0–$s7)`, `$24–$25 ($t8–$t9)`, `$31 ($ra)`. Caption: "32 × 32-bit register file".

- Memory strip: boxes labelled 0, 4, 8, 12, 16, each 4 bytes wide. Caption: "byte addressed; a word = 4 bytes; word addresses are multiples of 4".
- Instruction bar: 32 cells numbered 31 on the left to 0 on the right.

Narration:

> Three quick ideas before we start. One: an **instruction** is a single command to the processor, such as "add these two numbers". In MIPS, every instruction is exactly **32 bits** long. Two: a **register** is a very fast storage location inside the processor. MIPS has **32 registers**, each 32 bits wide. A 32-bit quantity is called a **word**. Registers are numbered 0 to 31, and they also have assembler names. The notes tell us: $t0 to $t7 are registers 8 to 15, $s0 to $s7 are registers 16 to 23, and $t8, $t9 are registers 24 and 25. Register 0, called $zero, always holds zero. Three: **memory** is much larger but slower. MIPS memory is **byte addressed** — every address names one 8-bit byte. A word takes 4 bytes, so word addresses are multiples of 4: 0, 4, 8, 12 and so on. Finally, the **Program Counter**, or PC, is a register that holds the address of the instruction currently being executed.

On-screen revision points:

- Instruction = 32 bits. Word = 32 bits = 4 bytes.
- 32 registers; *t*0–t7 = 8–15, *s*0–s7 = 16–23, *t*8–t9 = 24–25.
- Memory is byte addressed; word addresses are multiples of 4.
- PC = address of the current instruction.

Scene 0.3 — Minimum Background: The Three Instruction Formats (preview)

Visual directions: Three coloured bars stack vertically. Each field slides in with its bit width. A running total counter on the right adds up to 32.

Diagram instructions:

```
R-format:  | op 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6 |   6+5+5+5+5+6 = 32
I-format:  | op 6 | rs 5 | rt 5 |   constant or address 16  |   6+5+5+16   = 32
J-format:  | op 6 |            target address 26              |   6+26       = 32
```

Bit positions printed under each bar: R: 31–26, 25–21, 20–16, 15–11, 10–6, 5–0. I: 31–26, 25–21, 20–16, 15–0. J: 31–26, 25–0.

Narration:

> Addressing modes depend on where each piece of information sits inside the 32-bit instruction, so here is a preview of the three MIPS formats. We study them field by field in Topic 9. **R-format** — "R" for register — has six fields: op, 6 bits; rs, 5 bits; rt, 5 bits; rd, 5 bits; shamt, 5 bits; and funct, 6 bits. Six plus five plus five plus five plus five plus six is thirty-two. **I-format** — "I" for immediate — has op, 6 bits; rs, 5; rt, 5; and a 16-bit constant or address. Six plus five plus five plus sixteen is thirty-two. **J-format** — "J" for jump — has a 6-bit opcode and a 26-bit target. Six plus twenty-six is thirty-two. Why are register fields 5 bits? Because 2 to the power 5 is 32, and we have exactly 32 registers.

On-screen revision points:

- R: 6-5-5-5-5-6. I: 6-5-5-16. J: 6-26. All = 32 bits.
- 5-bit register field because $2^5 = 32$ registers.

Scene 1.1 — What Is an Addressing Mode?

Visual directions: Heading "TOPIC 1 — MIPS ADDRESSING MODES". Then the definition from the notes appears word by word.

Diagram instructions: A question mark labelled "operand?" with three arrows pointing to three possible homes: "inside the instruction", "in a register", "in memory". A fourth arrow labelled "next instruction?" points to "instruction memory".

Narration:

> Every instruction needs data to work on. That data is called an **operand**. The question is: where is the operand, and how does the processor find it? The notes define it like this: "Multiple forms of addressing are generically called addressing modes. The method used to identify the location of an operand." So an **addressing mode** is the rule the processor uses to locate an operand — or, for branch and jump instructions, to locate the next instruction. The operand could be written directly inside the instruction. It could be in a register. Or it could be in memory, and we must calculate its address. MIPS has exactly five addressing modes. Let us list them, exactly as the notes do.

On-screen revision points:

- **Addressing mode**: the method used to identify the location of an operand.
- MIPS has 5 addressing modes.

Scene 1.2 — The Five MIPS Addressing Modes (overview)

Visual directions: The five-mode list from the notes appears, then the notes' five-part diagram (page 12) is redrawn cleanly.

Diagram instructions: Recreate the notes' figure exactly, with the operand box shaded blue:

1. **Immediate addressing**: `| op | rs | rt | Immediate |` — the Immediate field is shaded (operand is inside the instruction).
2. **Register addressing**: `| op | rs | rt | rd | … | funct |` — an arrow from the rs/rt/rd field to a box "Registers → Register" (shaded).
3. **Base addressing**: `| op | rs | rt | Address |` — rs selects a "Register" box; Register and Address both feed a circle "+"; output arrow to "Memory" with cells "Byte | Halfword | Word" (shaded).
4. **PC-relative addressing**: `| op | rs | rt | Address |` — "PC" box and Address feed a "+" circle; arrow to "Memory → Word" (shaded).

5. **Pseudodirect addressing**: `| op | Address |` — "PC" box and Address feed a circle containing ":" (concatenation); arrow to "Memory → Word" (shaded).

Narration:

> Here are the five modes, in the order of the notes. One — **Immediate addressing**, where the operand is a constant within the instruction itself. Two — **Register addressing**, where the operand is a register. Three — **Base or displacement addressing**, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction. Four — **PC-relative addressing**, where the branch address is the sum of the PC and a constant in the instruction. Five — **Pseudodirect addressing**, where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC. Look at the diagram. The shaded box in each row is where the operand lives. In mode 1, the operand is 16 bits of the instruction itself. In mode 2, it is a register. In mode 3, it is in memory — and it can be a byte, a halfword or a word. Modes 4 and 5 are different: they do not find data. They find **instructions** in memory — they tell the processor where to go next. Now let us study each one in detail with actual numbers.

On-screen revision points: | # | Mode | Operand is… | Used by | |---|---|---|---| | 1 | Immediate | 16-bit constant inside instruction | addi, slti, andi, ori | | 2 | Register | a register | add, sub, and, or, slt | | 3 | Base / displacement | memory at [register + constant] | lw, sw | | 4 | PC-relative | instruction at [PC + 4 + offset×4] | beq, bne | | 5 | Pseudodirect | instruction at [PC upper 4 bits ‖ 26 bits ‖ 00] | j, jal |

Scene 1.3 — Mode 1: Immediate Addressing — Definition and Format

Visual directions: Heading "1. Immediate Addressing". The I-format bar appears; the 16-bit field glows blue with the label "the operand itself".

Diagram instructions:

```
addi  $s3, $s3, 4
| op = 8 | rs = 19 ($s3) | rt = 19 ($s3) | immediate = 4 |
|  6     |      5        |      5        |      16       |
```

Arrows: rs → "Register file read" → ALU input A. Immediate → "Sign-extend 16→32" → ALU input B. ALU output → back to register rt.

Narration:

> Immediate addressing. **Definition:** the operand is a constant within the instruction itself. "Immediate" means "immediately available" — the processor does not need to go to a register or to memory to fetch it. It is already sitting inside the instruction that was just fetched. Immediate instructions use the **I-format**. The 16-bit field at the right holds the constant. The rs field names the source register, and the rt field names the destination register. The notes give the example: `addi $s3, $s3, 4`. This means: $s3 gets $s3 plus 4. The "i" in addi stands for "immediate". Step by step: Step one — the instruction is fetched.

> Step two — the processor reads register rs, which is $s3. Step three — the 16-bit constant 4 is taken straight from the instruction and sign-extended to 32 bits so the ALU can use it. Step four — the ALU adds them. Step five — the result is written back into register rt, which is again $s3.

On-screen revision points:

- Immediate: operand = constant inside the instruction. Format: I.
- Constant is 16 bits, range -2^{15} to $+2^{15} - 1$, i.e. −32768 to +32767. (*Notes print "−215 to +215 − 1"; it means 2^{15}.*)

Scene 1.4 — Immediate Addressing — Worked Example with Encoding

Visual directions: Calculator-style panel. Values appear line by line.

Diagram instructions: A two-row register table "before / after" for $s3, and the 32-bit binary encoding split into coloured fields.

Worked example:

```
Instruction:  addi $s3, $s3, 4
Given:        $s3 = 20 before execution

Operand calculation:
  Operand 1 = contents of rs = $s3 = 20        (register addressing)
  Operand 2 = immediate field = 4               (immediate addressing)
  Result    = 20 + 4 = 24  →  $s3 = 24

Encoding:
  op = 8   → 001000
  rs = 19  → 10011
  rt = 19  → 10011
  imm = 4  → 0000 0000 0000 0100
  32 bits: 001000 10011 10011 0000000000000100
  Group in 4s: 0010 0010 0111 0011 0000 0000 0000 0100
  Hex:         0x22730004
```

Narration:

> Let us put numbers in. Suppose $s3 holds 20. The first operand comes from register $s3 — that is 20. The second operand is written inside the instruction — that is 4. The ALU computes 20 plus 4 equals 24, and $s3 now holds 24. Now let us see how this instruction is stored as bits. The opcode for addi is 8, which in 6 bits is 0 0 1 0 0 0. $s3 is register 19, which in 5 bits is 1 0 0 1 1. It goes in both rs and rt. The constant 4 in 16 bits is thirteen zeros followed by 1 0 0. Put all the fields together and group them in fours, and the instruction in hexadecimal is 2 2 7 3 0 0 0 4. Now, a special point from the notes: MIPS has **no subtract-immediate instruction**. To subtract a constant, you just use a negative constant. The notes give `addi $s2, $s1, -1`. If $s1 = 10, then $s2 = 10 + (−1) = 9. The

> 16-bit field holds −1 in two's complement, which is 1111 1111 1111 1111, hex FFFF. When sign-extended to 32 bits it becomes FFFF FFFF, which is still −1.

On-screen revision points:

- `addi $s3,$s3,4` → 0x22730004.
- No `subi`: use `addi $s2,$s1,-1`. −1 in 16 bits = 0xFFFF.
- Exam tip: be able to identify which field holds the immediate (bits 15–0).

Scene 1.5 — Mode 2: Register Addressing — Definition and Format

Visual directions: Heading "2. Register Addressing". R-format bar appears; the rs, rt, rd fields each shoot an arrow into the register file.

Diagram instructions:

```
add $t0, $s1, $s2
| op=0 | rs=17 ($s1) | rt=18 ($s2) | rd=8 ($t0) | shamt=0 | funct=32 |
```

Register file drawn with Read register 1 ← rs, Read register 2 ← rt, Write register ← rd. Read data 1 and Read data 2 → ALU. ALU result → Write data.

Narration:

> Register addressing. **Definition:** the operand is a register. All the operands — both sources and the destination — are registers. The instruction does not contain the data; it contains the **register numbers**, 5 bits each. This mode is used by the arithmetic and logical R-format instructions: add, sub, and, or, and slt. Take `add $t0, $s1, $s2`. In words: $t0 gets $s1 plus $s2. rs, the first source, is $s1, which is register 17. rt, the second source, is $s2, register 18. rd, the destination, is $t0, register 8. Step by step: the processor uses rs and rt to read two registers, the ALU adds the two values, and the result is written to the register named by rd. Register addressing is the fastest mode, because registers are inside the processor. That is why the notes say registers are "used for frequently accessed data".

On-screen revision points:

- Register addressing: operand is in a register. Format: R.
- rs = 1st source, rt = 2nd source, rd = destination.

Scene 1.6 — Register Addressing — Worked Example with Encoding

Visual directions: Register table before/after; the notes' encoding table for `add $t0,$s1,$s2` reproduced exactly.

Worked example:

```
Given: $s1 = 15, $s2 = 27
add $t0, $s1, $s2
  rs = $s1 = 15,  rt = $s2 = 27
  $t0 = 15 + 27 = 42

Encoding (from the notes):
  field:   op      rs     rt     rd     shamt  funct
  name:    special $s1    $s2    $t0    0      add
  decimal: 0       17     18     8      0      32
  binary:  000000  10001  10010  01000  00000  100000

  32 bits: 0000 0010 0011 0010 0100 0000 0010 0000 = 0x02324020
```

Narration:

> Suppose $s1 holds 15 and $s2 holds 27. Then $t0 receives 15 plus 27, which is 42. Here is the encoding exactly as in the notes. The op field is 0 — all R-format instructions have opcode 0, which the notes call "special". rs is 17, rt is 18, rd is 8, shamt is 0, and funct is 32, which means "add". In binary: 000000, 10001, 10010, 01000, 00000, 100000. Grouping in fours gives hexadecimal 0 2 3 2 4 0 2 0. Important exam point: add and sub have the **same opcode**, 0. The hardware uses the **funct** field to tell them apart: funct 32 is add, funct 34 is subtract.

On-screen revision points:

- `add $t0,$s1,$s2` = 0x02324020.
- R-format op = 0; funct: add = 32, sub = 34.

Scene 1.7 — Mode 3: Base or Displacement Addressing — Definition

Visual directions: Heading "3. Base or Displacement Addressing". Notes' diagram row 3 is animated: register value and 16-bit address slide into the "+" circle; the sum shoots an arrow into memory.

Diagram instructions:

```
lw $t0, 32($s3)
| op=35 | rs=19 ($s3) base | rt=8 ($t0) dest | address/offset = 32 |
     $s3 contents ──┐
                   (+) ──► Effective Address ──► Data Memory ──► $t0
 sign-extend(32) ──┘
```

Label the two inputs of the "+" as "base register" and "offset/displacement", matching the notes' labels in the data-transfer example.

Narration:

> Base or displacement addressing. **Definition from the notes:** the operand is at the memory location whose address is the sum of a register and a constant in the instruction.

> The register is called the **base register**. The constant is called the **offset** or **displacement**. Their sum is the **effective address** — the actual memory address of the operand. This is the mode used by the load and store instructions, lw and sw, which use the I-format. In `lw $t0, 32($s3)`, the number outside the brackets, 32, is the offset. The register inside the brackets, $s3, is the base register. Step by step: Step one — read the base register, rs, which is $s3. Step two — take the 16-bit offset and sign-extend it to 32 bits. Step three — the ALU adds them. This sum is the effective address. Step four — read memory at that address. Step five — write the loaded word into rt, which is $t0. Notice something important: for lw, the **rt field is the destination**, not a source. The notes point this out specifically.

On-screen revision points:

- **Effective Address = Base register (rs) + sign-extended 16-bit offset.**
- Used by lw, sw (and byte/halfword loads and stores). Format: I.
- In lw, rt = destination register.

Scene 1.8 — Base Addressing — Worked Example: Array Access

Visual directions: A memory strip showing array A starting at address 1000: A[0] at 1000, A[1] at 1004, … A[8] at 1032. A[8] is highlighted.

Worked example (from the notes' data-transfer example, with values added):

```
C code:   g = h + A[8];
          g in $s1, h in $s2, base address of A in $s3

Why offset 32?  Index 8 × 4 bytes per word = 32

Given: $s3 = 1000 (base address of A), A[8] = 50, $s2 (h) = 7

lw  $t0, 32($s3)
    Effective address = $s3 + 32 = 1000 + 32 = 1032
    $t0 = Memory[1032] = A[8] = 50
add $s1, $s2, $t0
    $s1 = 7 + 50 = 57   →  g = 57

Encoding of lw $t0, 32($s3):
  op=35   100011
  rs=19   10011
  rt=8    01000
  imm=32  0000 0000 0010 0000
  → 1000 1110 0110 1000 0000 0000 0010 0000 = 0x8E680020
```

Narration:

> This is the example from the notes. In C, we have g equals h plus A of 8. g is in $s1, h is in $s2, and the starting address of array A is in $s3. Why is the offset 32 and not 8? Because memory is byte addressed and each array element is a 4-byte word. Element 8 is 8 times 4, which is 32 bytes from the start. The notes say: "Index 8 requires offset of 32." Now

> numbers: let $s3 be 1000. The effective address is 1000 plus 32 equals 1032. Memory at 1032 holds A of 8, which is 50, so $t0 gets 50. Then `add $s1, $s2, $t0` gives 7 plus 50 equals 57. The encoding: opcode 35 for lw, rs 19 for $s3, rt 8 for $t0, and 32 in the address field. The notes say exactly this: "19 for $s3 is placed in the rs field, 8 for $t0 is placed in the rt field, and 32 is placed in the address field." In hex the instruction is 8 E 6 8 0 0 2 0.

On-screen revision points:

- Byte offset = index × 4.
- `lw $t0,32($s3)` → op 35, rs 19, rt 8, address 32 → 0x8E680020.

Scene 1.9 — Base Addressing — Store Word Example

Visual directions: Same memory strip; now A[12] at 1048 is highlighted and an arrow goes from register to memory (opposite direction to lw).

Worked example (notes' Example 2):

```
C code:   A[12] = h + A[8];
          h in $s2, base address of A in $s3

lw  $t0, 32($s3)     # $t0 = A[8]
add $t0, $s2, $t0    # $t0 = h + A[8]
sw  $t0, 48($s3)     # A[12] = $t0      (12 × 4 = 48)

Given $s3 = 1000, A[8] = 50, h = 7:
  lw : EA = 1000 + 32 = 1032 → $t0 = 50
  add: $t0 = 7 + 50 = 57
  sw : EA = 1000 + 48 = 1048 → Memory[1048] = 57  (A[12] = 57)

Encoding of sw $t0, 48($s3):
  op=43 101011 | rs=19 10011 | rt=8 01000 | imm=48 0000 0000 0011 0000
  → 1010 1110 0110 1000 0000 0000 0011 0000 = 0xAE680030
```

Narration:

> Store word uses exactly the same addressing mode, but the data flows the other way — from a register into memory. The notes' second example is: A of 12 equals h plus A of 8. We load A of 8, add h, and then store the result into A of 12. The offset for element 12 is 12 times 4, which is 48. With our values, the store's effective address is 1000 plus 48, which is 1048, and the value 57 is written there. For sw, the rt field names the register whose value is **stored** — so here rt is a source, not a destination. Exam point: the opcode for lw is 35 and for sw is 43, as given in the notes' instruction table.

On-screen revision points:

- sw: Memory[rs + offset] ← rt. lw: rt ← Memory[rs + offset].
- Opcodes: lw = 35, sw = 43, addi = 8, R-type = 0.

Scene 1.10 — Mode 4: PC-Relative Addressing — Definition

Visual directions: Heading "4. PC-Relative Addressing". Animated program listing with the PC arrow moving down instruction by instruction, then jumping.

Diagram instructions:

```
beq rs, rt, offset
| op=4 | rs | rt | 16-bit offset (in words) |

PC ──► [+4] ──► PC+4 ──┐
                       (+) ──► Branch target address
offset ──► sign-extend ──► shift left 2 ──┘
```

Narration:

> PC-relative addressing. **Definition from the notes:** the branch address is the sum of the PC and a constant in the instruction. This mode is used by conditional branches — beq, "branch if equal", and bne, "branch if not equal". They use the I-format. Why "relative"? Because the target is not given as a full address. It is given as a distance from where we are now. Most branches go to nearby instructions — in loops and if statements — so a 16-bit distance is enough. There are two details the notes emphasise, and both are exam favourites. Detail one: the base for the calculation is **the address of the instruction following the branch**, that is, **PC + 4**, not the PC of the branch itself. Detail two: the offset is a **word offset**. It is **shifted left by 2 bits**, which multiplies it by 4, turning a count of instructions into a count of bytes. This increases the range of the branch by a factor of 4. So the formula is: branch target address equals PC plus 4, plus the sign-extended offset shifted left by 2.

On-screen revision points:

- **Branch target = (PC + 4) + (sign-extend(offset) << 2)**
- Offset counts **instructions (words)**, not bytes.
- Branch taken → PC = target. Not taken → PC = PC + 4.

Scene 1.11 — PC-Relative Addressing — Worked Examples

Visual directions: Program listing with addresses in hex on the left. Arrow animation shows the branch jumping forward, then a second example jumping back.

Worked example 1 — forward branch:

```
Address       Instruction
0x00400010    beq $s3, $s4, Else       # offset field = 3
0x00400014    add $s0, $s1, $s2
0x00400018    j   Exit
0x0040001C    ...
0x00400020    Else: sub $s0, $s1, $s2

PC of beq          = 0x00400010
PC + 4             = 0x00400014
offset field       = 3  (0000 0000 0000 0011)
sign-extend        = 0x00000003
shift left 2       = 3 × 4 = 12 = 0x0000000C
Target             = 0x00400014 + 0x0000000C = 0x00400020  ✔ (Else)

If $s3 = $s4  → branch taken  → PC = 0x00400020
If $s3 ≠ $s4  → not taken     → PC = 0x00400014
```

Worked example 2 — backward branch (a loop):

```
offset field = −2 → 16 bits: 1111 1111 1111 1110 (0xFFFE)
sign-extend  → 0xFFFFFFFE  (= −2)
shift left 2 → −8
PC of branch = 0x00400010, PC + 4 = 0x00400014
Target = 0x00400014 − 8 = 0x0040000C
```

Narration:

> Example one. The beq is at address hex 0040 0010. PC plus 4 is 0040 0014. The offset field holds 3. Sign-extending keeps it as 3. Shifting left by 2 multiplies by 4, giving 12, which is hex C. Hex 0040 0014 plus C is 0040 0020 — exactly the address of the label Else. Notice: counting from the instruction **after** the branch, Else is 3 instructions away — 0014, 0018, 001C, then 0020. That is why the offset is 3. Example two, a backward branch, as in a loop. The offset is −2, stored in two's complement as hex FFFE. Sign extension copies the 1 sign bit into the upper 16 bits, giving FFFF FFFE, which is still −2. Shift left 2 gives −8. 0040 0014 minus 8 is 0040 000C. Sign extension is what makes backward branches possible. One more note: the notes mention that in the real MIPS instruction set, **branches are delayed** — the instruction right after the branch is always executed. For simplicity, the notes and this video ignore delayed branches.

On-screen revision points:

- Offset = number of instructions from (PC + 4) to the target.
- Negative offset → backward branch (loops); sign extension is essential.
- Delayed branch: following instruction always executes (ignored in this design).

Scene 1.12 — Mode 5: Pseudodirect Addressing — Definition

Visual directions: Heading "5. Pseudodirect Addressing". J-format bar. Then a 32-bit address bar assembled from three coloured pieces.

Diagram instructions:

```
j target
| op = 2 (6 bits) | target (26 bits) |

New PC (32 bits) = | PC[31:28] (4 bits) | target (26 bits) | 00 (2 bits) |
                         4      +      26      +      2       = 32
```

The ":" circle from the notes' diagram is labelled "concatenation (join side by side) — NOT addition".

Narration:

> Pseudodirect addressing. **Definition from the notes:** the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC. This mode is used by the jump instruction, j, and jump-and-link, jal. They use the **J-format**: a 6-bit opcode and a 26-bit target. The notes give the exact rule: PC gets PC bits 31 to 28, followed by IR bits 25 to 0, followed by 0 0. "IR" is the instruction register — the instruction being executed — so IR bits 25 to 0 are the 26-bit target field. Let us count the bits: 4 bits from the PC, plus 26 bits from the instruction, plus 2 zero bits — 32 bits, a complete address. Very important: this is **concatenation**, not addition. The pieces are placed side by side. That is why the notes' diagram shows a colon, not a plus sign. Why "pseudo" direct? A direct address would be a full 32-bit address in the instruction. We cannot fit 32 bits, because 6 bits are used by the opcode. So we get "almost" direct — 26 bits from the instruction and the rest borrowed from the PC. Why append two zeros? Because every instruction address is a multiple of 4, its lowest two bits are always 0 0. There is no need to store them. Appending 0 0 is the same as shifting left by 2.

On-screen revision points:

- **Jump address = PC[31:28] ‖ target[25:0] ‖ 00** (‖ = concatenation).
- 4 + 26 + 2 = 32 bits. Jump range: within the same 256 MB (2^{28} bytes) region.
- *Clarification beyond the notes: the textbook strictly uses the upper 4 bits of PC + 4; the notes write PC[31:28]. The result differs only at a 256 MB boundary.*

Scene 1.13 — Pseudodirect Addressing — Worked Example

Visual directions: Three-row binary assembly animation.

Worked example:

```
j instruction at PC = 0x00400018, target field = 0x0100008

target (26 bits)  = 00 0001 0000 0000 0000 0000 1000
shift left 2      = 0000 0100 0000 0000 0000 0010 0000  (28 bits) = 0x0400020
PC[31:28]         = 0000  (from 0x0...)
Concatenate       = 0000 | 0000 0100 0000 0000 0000 0010 0000
New PC            = 0x00400020
```

Narration:

> Suppose a jump at address hex 0040 0018 has the target field hex 010 0008. Write the 26 bits out. Shift left by 2 — that is, append two zeros — and we get the 28-bit value hex 040 0020. The upper 4 bits of the PC are 0 0 0 0. Placing them in front gives hex 0040 0020. The processor loads this into the PC and continues execution there. Quick check: hex 0040 0020 divided by 4 is hex 0010 0008 — exactly our target field. So the target field is simply the word address of the destination.

On-screen revision points:

- Target field = (destination address ÷ 4), lower 26 bits.
- `j` opcode = 2 (J-format).

Scene 1.14 — Comparing the Five Addressing Modes

Visual directions: Full-screen comparison table builds row by row; then a "How to identify the mode" decision flow chart.

Diagram instructions: Decision flowchart:

```
Is it a branch (beq/bne)?        → PC-relative
Is it a jump (j/jal)?            → Pseudodirect
Is there "number(register)"?     → Base/displacement
Is there a plain constant?       → Immediate (+ register for the other operand)
Only registers?                  → Register
```

Narration:

> Let us compare all five. Immediate and register addressing find **data** that is immediately available — inside the instruction, or in a register. No memory access is needed. Base addressing finds **data in memory** using register plus offset. PC-relative and pseudodirect find **instructions** — the next place to execute. PC-relative **adds** an offset to PC + 4. Pseudodirect **concatenates** bits. PC-relative uses a 16-bit offset; pseudodirect uses a 26-bit target, so jumps reach much further than branches. One more subtle point: a single instruction can use **more than one** mode. `addi $s3, $s3, 4` uses register addressing for $s3 and immediate addressing for 4. `beq $s1, $s2, L` uses register addressing to compare and PC-relative addressing for the target. And `lw` uses register addressing for the destination and base addressing for the memory operand. Here is a simple way to identify the mode: look at the instruction. Branch? PC- relative. Jump? Pseudodirect. A number followed by a register in brackets? Base. A plain constant? Immediate. Only registers? Register addressing.

On-screen revision points: | Mode | Format | Address / operand calculation | Memory access? | |---|---|---|---| | Immediate | I | operand = sign-extend(imm16) | No | | Register | R | operand = Reg[rs], Reg[rt] | No | | Base | I | EA = Reg[rs] + sign-extend(offset16) | Yes (data) | | PC-relative | I | target = PC + 4 + (sign-extend(offset16) << 2) | Fetches instruction | | Pseudodirect | J | target = PC[31:28] ‖ addr26 ‖ 00 | Fetches instruction |

Exam points for Topic 1:

- List and define all 5 modes in the notes' words; draw the notes' 5-part diagram.
- PC-relative uses PC + 4 and a word offset (shift left 2).
- Pseudodirect = concatenation, not addition.
- lw/sw: rt is destination (lw) or source (sw).

End of Part 1. Continue with Part 2 — A Basic MIPS Implementation.

PART 2 — Topic 2: A Basic MIPS Implementation

Scene 2.1 — The MIPS Subset We Implement

Visual directions: Heading "TOPIC 2 — A BASIC MIPS IMPLEMENTATION". Three coloured category cards appear, exactly as listed in the notes.

Diagram instructions:

- Card 1 (blue): "Memory-reference: `lw`, `sw`"
- Card 2 (green): "Arithmetic-logical: `add`, `sub`, `AND`, `OR`, `slt`"
- Card 3 (orange): "Branch `beq` and jump `j` (added last)"
- Small grey card: "Not included: shift, multiply, divide, floating point"

Narration:

> Now that we know how instructions find their operands, let us see how a processor actually executes them. The notes examine an implementation of a **subset** of the core MIPS instruction set. It has three groups. First, the **memory-reference** instructions: load word, lw, and store word, sw. Second, the **arithmetic-logical** instructions: add, sub, AND, OR, and slt — "set on less than". Third, the instructions **branch equal**, beq, and **jump**, j, which are added last. The notes are clear that this subset does not include all integer instructions — shift, multiply and divide are missing — and no floating-point instructions. But it illustrates the key principles used in creating a datapath and designing the control. The remaining instructions are implemented in a similar way. Two new words: a **datapath** is the hardware that moves and processes data — memories, registers, ALU, adders and the wires between them. **Control** is the logic that tells the datapath what to do for each instruction.

On-screen revision points:

- Subset: lw, sw | add, sub, and, or, slt | beq, j.
- Datapath = hardware that operates on/holds data. Control = tells it what to do.

Scene 2.2 — The First Two Steps Are Identical for Every Instruction

Visual directions: Two large animated steps. A PC box sends an address to instruction memory; the instruction comes out and its register fields light up.

Diagram instructions:

```
Step 1:  PC ──address──► [Instruction memory] ──instruction──►
Step 2:  instruction[25:21] (rs) ──► Read register 1 ─┐
         instruction[20:16] (rt) ──► Read register 2 ─┴► [Registers] ► Read data 1,
Read data 2
```

Narration:

> The notes say: for every instruction, the first two steps are identical. Step one: send the **program counter** to the memory that contains the code, and **fetch** the instruction from that memory. Step two: **read one or two registers**, using fields of the instruction to select the registers to read. For load word, we need to read only one register — the base register. Most other instructions read two. After these two steps, the actions depend on the instruction class. But here is the good news: within each class, the actions are largely the same, regardless of the exact instruction. The simplicity and regularity of the MIPS instruction set simplifies the implementation.

On-screen revision points:

- Step 1: Fetch — PC → instruction memory → instruction.
- Step 2: Read registers — fields of the instruction select 1 or 2 registers.

Scene 2.3 — Every Class (Except Jump) Uses the ALU

Visual directions: ALU symbol centre-screen. Three arrows labelled with the three uses.

Narration:

> After reading the registers, all instruction classes except jump use the arithmetic-logical unit, the ALU — but for different purposes. Memory-reference instructions use the ALU for an **address calculation** — base plus offset, the base addressing we just learned. Arithmetic-logical instructions use it for the **operation execution** — the add, subtract, and, or, or comparison itself. Branches use it for **comparison** — are the two registers equal? After the ALU, the classes differ again. A memory-reference instruction must access the data memory — to read data for a load, or write data for a store. An arithmetic-logical or load instruction must write the data, from the ALU or the memory, back into a register. And for a branch, we may need to change the next instruction address based on the comparison; otherwise the PC is incremented by 4 to get the address of the next instruction.

On-screen revision points: | Class | ALU used for | After the ALU | |---|---|---| | lw / sw | address calculation | access data memory (lw also writes register) | | R-type | operation |

write result to register | | beq | comparison (subtract, check Zero) | choose PC+4 or branch target | | j | not used | PC ← jump address |

Scene 2.4 — The Five Phases of Execution

Visual directions: A horizontal 5-stage strip: Fetch → Register read/Decode → ALU → Memory → Write-back, plus a separate "PC update" arrow looping back.

Narration:

> Let us give names to these steps; we will use them for every instruction. One — **instruction fetch**: read the instruction at address PC. Two — **register read**: read the registers named by rs and rt. At the same time the control unit looks at the opcode and decides what to do — this is decoding. Three — **ALU operation**: compute a result, an address, or a comparison. Four — **memory access**: only for lw and sw. Five — **write-back**: write a result into the register file — only for R-type and lw. And throughout — **PC update**: the PC becomes PC + 4, or the branch target, or the jump address. In this first design, the notes say every instruction begins execution on one clock edge and completes on the next clock edge. So all five phases happen in a **single clock cycle**. We call this a **single-cycle implementation**.

On-screen revision points:

- Fetch → Register read/decode → ALU → Memory → Write-back; PC update.
- Single-cycle: each instruction completes in one clock cycle.

Scene 2.5 — High-Level View: Figure 1.1

Visual directions: Redraw the notes' Figure 1.1, then animate data flow.

Diagram instructions (Figure 1.1, clean redraw):

- Left: **PC** box → "Address" of **Instruction memory**; "Instruction" output splits into three "Register #" inputs of **Registers** and a branch to the **Data** input region.
- Top: an **Add** with inputs PC and constant **4**; a second **Add** with inputs (PC + 4) and the offset from the instruction. Both adder outputs loop back to PC.
- **Registers** outputs → **ALU** → "Address" of **Data memory**.
- Registers' second output → "Data" (write data) of Data memory.
- Data memory output and ALU output both loop back to the "Data" input of Registers.
- Thick lines labelled "buses"; dots where crossing lines connect.

Narration:

> This is Figure 1.1 from the notes — an abstract view of the implementation, showing the major functional units and the connections between them. All instructions start by using the program counter to supply the instruction address to the instruction memory. After the instruction is fetched, the register operands are specified by fields of that instruction. Once the register operands have been fetched, they can be used to compute a memory

> address for a load or store, an arithmetic result for an arithmetic- logical instruction, or a compare for a branch. For an arithmetic-logical instruction, the ALU result is written to a register. For a load or store, the ALU result is used as an address — to store a value from the registers, or to load a value from memory into the registers. Branches use the ALU output to decide the next instruction address, which comes either from the adder where PC and branch offset are summed, or from the adder that increments the current PC by 4. The thick lines are **buses** — groups of many signals, here usually 32 wires. The arrows show the direction of information. Where crossing lines are connected, a dot is drawn.

Scene 2.6 — What Figure 1.1 Leaves Out

Visual directions: Figure 1.1 with three red circles where two wires meet one input, and three red question marks next to units that need control.

Narration:

> The notes point out two omissions in Figure 1.1. **First**, in several places, data going to a unit comes from two different sources. The value written into the PC can come from one of two adders. The data written into the register file can come from the ALU or from data memory. And the second ALU input can come from a register or from the immediate field. You cannot simply wire two sources together. We need a logic element that chooses one of them — a **multiplexor**, which the notes say might better be called a **data selector**. It selects among its inputs based on its control lines. **Second**, several units must be controlled depending on the instruction type. The data memory must read on a load and write on a store. The register file must be written on a load and on an arithmetic-logical instruction. The ALU must perform one of several operations. These control lines are set based on fields in the instruction.

On-screen revision points:

- Omission 1 → add **multiplexors (data selectors)**.
- Omission 2 → add **control lines** set from the instruction.

Scene 2.7 — Figure 1.2: Adding Multiplexors and Control

Visual directions: Redraw notes' Figure 1.2. Control lines drawn in blue. Highlight each multiplexor in turn.

Diagram instructions (Figure 1.2):

- Everything in Figure 1.1, plus:
- **Top Mux**: inputs (PC + 4) and (branch target); output → PC. Its select comes from an **AND gate** whose inputs are **Branch** (from Control) and **Zero** (from ALU).
- **Middle Mux** (right of data memory): inputs ALU result and Data memory output; output → register file "Data" input.
- **Bottom Mux** (before ALU): inputs Register read data 2 and instruction offset field; output → ALU second input.

- **Control** ellipse at the bottom, input = instruction, blue outputs: **RegWrite**, **ALU operation**, **MemRead**, **MemWrite**, **Branch**, and mux selects.

Narration:

> Figure 1.2 is the same datapath with three multiplexors added, plus control lines for the major functional units. A **control unit**, which has the instruction as its input, determines how to set the control lines for the functional units and two of the multiplexors. The **top multiplexor** controls what value replaces the PC: PC + 4 or the branch destination address. It is controlled by a gate that ANDs together the **Zero** output of the ALU and a control signal indicating the instruction is a **Branch**. The **middle multiplexor** steers either the ALU output — for an arithmetic- logical instruction — or the data memory output — for a load — into the register file. The **bottom multiplexor** decides whether the second ALU input comes from the registers — for arithmetic-logical or branch — or from the offset field of the instruction — for a load or store. The remaining control lines decide the ALU operation, whether data memory reads or writes, and whether the registers perform a write. Because MIPS is regular and simple, a simple decoding process is enough to set these control lines.

On-screen revision points:

- Top mux → PC source; select = Branch AND Zero.
- Middle mux → register write data: ALU result or memory data.
- Bottom mux → ALU input 2: register or sign-extended offset.

Scene 2.8 — Control Signals Explained (Clarification beyond the notes)

Visual directions: Tag in the corner: "Clarification — beyond the notes (standard textbook values)". Table builds row by row.

Diagram instructions: Table of signals with a one-line meaning each.

Narration:

> The notes name the control lines in their figures but do not tabulate their values. To explain every instruction fully, I will use the standard names and values from the textbook the notes are drawn from, Patterson and Hennessy. I am marking this clearly as a clarification. **RegDst** — chooses the destination register number: rt when 0, rd when 1. **ALUSrc** — chooses the ALU's second input: register read data 2 when 0, the sign-extended immediate when 1. This is the notes' "bottom multiplexor". **MemtoReg** — chooses what is written into the register: ALU result when 0, memory data when 1. This is the "middle multiplexor". **RegWrite** — when 1, the register file is written at the clock edge. **MemRead** — when 1, data memory is read. **MemWrite** — when 1, data memory is written. **Branch** — 1 for a beq. Together with Zero it forms **PCSrc**, the select of the top multiplexor. **ALU operation** — a 4-bit code telling the ALU what to do. **Jump** — 1 for j; selects the jump address for the PC.

On-screen revision points (clarification): | Signal | 0 means | 1 means | |---|---|---| | RegDst | write reg = rt | write reg = rd | | ALUSrc | ALU B = Read data 2 | ALU B = sign-extended imm | | MemtoReg | reg data = ALU result | reg data = memory data | | RegWrite | no write | write register | | MemRead | — | read data memory | | MemWrite | — | write data memory | | Branch | not beq | beq | | PCSrc = Branch AND Zero | PC = PC + 4 | PC = branch target |

ALU control codes (4 bits): AND 0000, OR 0001, add 0010, subtract 0110, set on less than 0111.

Scene 2.9 — Instruction Class 1: R-Type (ADD, SUB, AND, OR, SLT)

Visual directions: Complete datapath dims; only the R-type path lights up green. Step counter 1–5 animates in sync with narration.

Diagram instructions — highlighted path:

```
PC → Instruction memory → instruction
  [25:21] rs → Read register 1 → Read data 1 → ALU A
  [20:16] rt → Read register 2 → Read data 2 → ALUSrc mux (0) → ALU B
  [15:11] rd → RegDst mux (1) → Write register
ALU result → MemtoReg mux (0) → Write data → register rd
PC → Add(+4) → PCSrc mux (0) → PC
```

Data memory and sign-extend shown greyed out.

Narration:

> Let us execute our first instruction class: R-type, also called arithmetic- logical instructions. The notes say they all read two registers, perform an ALU operation on the contents, and write the result to a register. **Instruction:** `add $t1, $t2, $t3` — the typical example in the notes. It reads $t2 and $t3 and writes $t1. **Purpose:** $t1 gets $t2 plus $t3. **Hardware involved:** PC, instruction memory, the PC + 4 adder, register file, ALU, and the multiplexors. **Data flow, step by step:** One — fetch: the PC addresses instruction memory, and the instruction comes out. At the same time, the adder computes PC + 4. Two — register read: bits 25 to 21, rs, select $t2 on read register 1. Bits 20 to 16, rt, select $t3 on read register 2. Both values appear on read data 1 and 2. Three — ALU: ALUSrc is 0, so the ALU's second input is read data 2. The ALU operation is "add". Four — memory: not used. MemRead and MemWrite are both 0. Five — write-back: MemtoReg is 0, so the ALU result goes to write data. RegDst is 1, so the write register is rd, bits 15 to 11 — that is $t1. RegWrite is 1, so at the clock edge $t1 is written. PC update: Branch is 0, so the PC becomes PC + 4. **For sub, and, or, slt** the path is identical. Only the ALU operation changes — and that is chosen using the funct field.

Worked example:

```
PC = 0x00400000, $t2 = 12, $t3 = 5
add $t1,$t2,$t3 → ALU: 12 + 5 = 17 → $t1 = 17
sub $t1,$t2,$t3 → ALU: 12 − 5 = 7  → $t1 = 7
and $t1,$t2,$t3 → 1100 AND 0101 = 0100 → $t1 = 4
or  $t1,$t2,$t3 → 1100 OR  0101 = 1101 → $t1 = 13
slt $t1,$t2,$t3 → is 12 < 5? No → $t1 = 0
slt $t1,$t3,$t2 → is 5 < 12? Yes → $t1 = 1
Next PC = 0x00400000 + 4 = 0x00400004
```

Control signals (clarification): RegDst 1, ALUSrc 0, MemtoReg 0, RegWrite 1, MemRead 0, MemWrite 0, Branch 0, ALU op = add/sub/and/or/slt from funct.

On-screen revision points:

- R-type: Reg[rd] ← Reg[rs] op Reg[rt]; PC ← PC + 4.
- slt rd, rs, rt: if (rs < rt) rd = 1 else rd = 0.

Scene 2.10 — Instruction Class 2a: Load Word (LW)

Visual directions: Load path lights up in blue — the longest path; a "longest path" badge appears.

Diagram instructions — highlighted path:

```
PC → Instruction memory
  [25:21] rs → Read register 1 → Read data 1 → ALU A
  [15:0] offset → Sign-extend (16→32) → ALUSrc mux (1) → ALU B
ALU result (address) → Data memory Address; MemRead = 1
Data memory Read data → MemtoReg mux (1) → Write data
  [20:16] rt → RegDst mux (0) → Write register
PC → Add(+4) → PC
```

Narration:

> **Instruction:** `lw $t0, 32($s3)`. **Purpose:** load the word at address $s3 + 32 into $t0. This is base addressing. **Hardware involved:** PC, instruction memory, PC adder, register file, sign-extension unit, ALU, data memory, and the multiplexors — load uses every major unit. **Data flow:** One — fetch the instruction; compute PC + 4. Two — read register rs, $s3. Only one register is needed. At the same time, the 16-bit offset, 32, is sign-extended to 32 bits. Three — ALUSrc is 1, so the ALU's second input is the sign-extended offset. The ALU adds: base plus offset equals the address. Four — the address goes to data memory with MemRead equal to 1. The memory outputs the word stored there. Five — MemtoReg is 1, so the memory data, not the ALU result, goes to the register file. RegDst is 0, so the destination is rt — $t0. RegWrite is 1. PC becomes PC + 4. Because lw passes through instruction memory, register file, ALU, data memory and back to the register file, it is the **longest path** — and in a single-cycle design, the clock must be long enough for it.

Worked example:

```
$s3 = 0x10010000, Memory[0x10010020] = 99
Sign-extend(32) = 0x00000020
ALU: 0x10010000 + 0x00000020 = 0x10010020
Data memory read → 99 → $t0 = 99
```

Control signals (clarification): RegDst 0, ALUSrc 1, MemtoReg 1, RegWrite 1, MemRead 1, MemWrite 0, Branch 0, ALU op = add.

On-screen revision points:

- lw: Reg[rt] ← Mem[Reg[rs] + sign-ext(offset)].
- Uses ALL five phases → longest instruction.

Scene 2.11 — Instruction Class 2b: Store Word (SW)

Visual directions: Store path lights up in purple. Write-back arrow is crossed out with a red X.

Diagram instructions — highlighted path:

```
  [25:21] rs → Read register 1 → Read data 1 → ALU A
  [20:16] rt → Read register 2 → Read data 2 → Data memory "Write data"
  [15:0]  → Sign-extend → ALUSrc mux (1) → ALU B
ALU result → Data memory Address; MemWrite = 1
No register write (RegWrite = 0)
```

Narration:

> **Instruction:** `sw $t0, 48($s3)`. **Purpose:** store the value in $t0 into memory at $s3 + 48. **Data flow:** One — fetch, and PC + 4. Two — read **two** registers: rs, $s3, the base; and rt, $t0, the data to store. Three — ALUSrc is 1; the ALU adds base and sign-extended offset to form the address. Four — the address goes to data memory, read data 2 goes to the "write data" input, and MemWrite is 1. The word is written. Five — there is **no write-back**. RegWrite is 0. RegDst and MemtoReg do not matter — we call them "don't care", shown as X.

Worked example:

```
$s3 = 0x10010000, $t0 = 57
Address = 0x10010000 + 48 (0x30) = 0x10010030
Memory[0x10010030] ← 57
```

Control signals (clarification): RegDst X, ALUSrc 1, MemtoReg X, RegWrite 0, MemRead 0, MemWrite 1, Branch 0, ALU op = add.

On-screen revision points:

- sw: Mem[Reg[rs] + sign-ext(offset)] ← Reg[rt]. No write-back.

Scene 2.12 — Instruction Class 3: Branch Equal (BEQ)

Visual directions: Branch path lights up in orange. Two parallel computations animate side by side: "compare" (ALU) and "target" (adder).

Diagram instructions — highlighted path (from notes' Figure 2.5):

```
  rs → Read data 1 → ALU A
  rt → Read data 2 → ALUSrc mux (0) → ALU B ; ALU op = subtract → Zero
  [15:0] → Sign-extend → Shift left 2 → Add (with PC + 4) → Branch target
PCSrc = Branch AND Zero → top mux → PC
```

Narration:

> **Instruction:** `beq $t1, $t2, offset`. **Purpose:** if $t1 equals $t2, go to the branch target; otherwise continue. The notes say the branch datapath must do **two operations**: compute the branch target address, and compare the register contents. **Data flow:** One — fetch, and compute PC + 4. Two — read rs and rt. Three — two things happen in parallel. The ALU **subtracts** rt from rs. If they are equal, the result is zero and the ALU's **Zero** output becomes 1. Meanwhile, a separate adder computes the branch target: PC + 4 plus the sign- extended offset shifted left 2. Four — no memory access. Five — no register write. PC update: the Branch control signal is 1. If Zero is also 1, the AND gate outputs 1, and the PC receives the branch target — the **branch is taken**. If Zero is 0, the PC receives PC + 4 — the **branch is not taken**. Notice that the shift left 2 unit needs no real hardware. As the notes explain, it is simply a routing of wires that adds two zero bits to the low end.

Worked example:

```
PC = 0x00400010, offset = 3, PC + 4 = 0x00400014
Target = 0x00400014 + (3 << 2) = 0x00400014 + 12 = 0x00400020

Case A: $t1 = 9, $t2 = 9 → ALU 9 − 9 = 0 → Zero = 1
        PCSrc = 1 AND 1 = 1 → PC = 0x00400020 (taken)
Case B: $t1 = 9, $t2 = 4 → ALU 9 − 4 = 5 → Zero = 0
        PCSrc = 1 AND 0 = 0 → PC = 0x00400014 (not taken)
```

Control signals (clarification): RegDst X, ALUSrc 0, MemtoReg X, RegWrite 0, MemRead 0, MemWrite 0, Branch 1, ALU op = subtract.

On-screen revision points:

- **Branch taken**: condition satisfied, PC becomes branch target.
- **Branch not taken (untaken)**: condition false, PC = address of next sequential instruction.
- Zero flag = 1 when ALU result is 0, i.e. rs = rt.

Scene 2.13 — Instruction Class 4: Jump (J)

Visual directions: Tag "Clarification — jump hardware is not drawn in the notes". A small addition to the datapath: a "Shift left 2" on instruction bits [25:0], a concatenation with PC+4[31:28], and a "Jump" mux in front of the PC.

Diagram instructions:

```
instruction[25:0] (26) → Shift left 2 → 28 bits ─┐
PC+4[31:28] (4) ──────────────────────────────────┴──► Jump address (32)
Jump mux: input 0 = output of branch mux, input 1 = jump address → PC
```

Narration:

> **Instruction:** `j L`. **Purpose:** unconditionally go to label L. The notes say the jump instruction operates by replacing the lower 28 bits of the PC with the lower 26 bits of the instruction shifted left by 2 bits. Simply concatenating 00 to the jump offset accomplishes this shift. This is pseudodirect addressing from Topic 1. **Data flow:** fetch the instruction; take bits 25 to 0; append two zeros to get 28 bits; place the upper 4 bits of the PC in front; and send this through a jump multiplexor into the PC. The ALU, registers and data memory are not used. Jump has no register reads, no ALU operation, no memory access and no write-back. RegWrite and MemWrite must be 0 so that nothing is changed by accident.

Worked example: PC = 0x00400018, target field = 0x0100008 → `0000 ‖ 0x0100008 << 2 = 0x00400020`.

Control signals (clarification): Jump 1, RegWrite 0, MemWrite 0, MemRead 0, Branch 0, others X.

Scene 2.14 — Summary Table of All Instruction Classes

Visual directions: Full-screen summary with coloured rows matching the path colours used earlier.

On-screen revision points: | Instr. | Fetch | Reg read | ALU | Memory | Write-back | Next PC | |---|---|---|---|---|---|---| | add/sub/and/or/slt | ✔ | rs, rt | operation | — | rd ← ALU | PC+4 | | lw | ✔ | rs | rs + offset | read | rt ← Mem | PC+4 | | sw | ✔ | rs, rt | rs + offset | write | — | PC+4 | | beq | ✔ | rs, rt | rs − rt (Zero) | — | — | Zero ? target : PC+4 | | j | ✔ | — | — | — | — | PC[31:28] ‖ addr ‖ 00 |

Control-signal truth table (clarification, Patterson & Hennessy): | Instr. | RegDst | ALUSrc | MemtoReg | RegWrite | MemRead | MemWrite | Branch | ALUOp | |---|---|---|---|---|---|---|---|---|---| | R-type | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 10 | | lw | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 00 | | sw | X | 1 | X | 0 | 0 | 1 | 0 | 00 | | beq | X | 0 | X | 0 | 0 | 0 | 1 | 01 |

Narration:

> Here is everything in one table. Look how much is shared: every instruction fetches the same way, most use the ALU, and only a few control signals change between classes. This

> is exactly what the notes mean by "the simplicity and regularity of the MIPS instruction set simplifies the implementation". Finally, a limitation from the notes: this single-cycle design is easy to understand but **not practical**, because the clock cycle must be stretched to fit the **longest** instruction — the load. Every instruction, even a simple add, must take that long. The fix — pipelining — is mentioned in the notes as a later topic.

Exam points for Topic 2:

- First two steps identical for all instructions: fetch, read registers.
- ALU used for: address (lw/sw), operation (R), comparison (beq).
- Three multiplexors and their purposes (notes' Figure 1.2).
- Single-cycle design is impractical: clock set by the longest instruction.

End of Part 2. Continue with Part 3 — Hardware Structure.

PART 3 — Topic 3: Hardware Structure of the Basic MIPS Processor

---

Scene 3.1 — Two Kinds of Hardware Elements

Visual directions: Heading "TOPIC 3 — HARDWARE STRUCTURE". Screen split into two columns: "Combinational elements" and "State elements".

Diagram instructions: Left column icons: ALU, adder, multiplexor, sign-extend. Right column icons: PC, register file, instruction memory, data memory. A clock waveform under the right column with rising edges marked.

Narration:

> In Topic 2 we watched data move. Now we look closely at each piece of hardware. The notes use the term **datapath element**: a unit used to operate on or hold data within a processor. In the MIPS implementation, the datapath elements include the instruction and data memories, the register file, the ALU, and adders. Datapath elements come in two kinds. **Combinational elements** compute an output purely from their current inputs — like the ALU, adders, multiplexors and the sign-extension unit. They have no memory. **State elements** store values — like the PC, register file and data memory. They are updated on a clock edge. The notes describe the PC, register file writes and data memory writes as **edge-triggered** — they change only at the clock edge. We will introduce the components one at a time, and then put them all together.

On-screen revision points:

- **Datapath element**: unit that operates on or holds data in a processor.
- Combinational: ALU, adder, mux, sign-extend. State: PC, registers, memories.

Scene 3.2 — Program Counter (PC)

Visual directions: Notes' Figure 2.1(b): a tall narrow box "PC" with one input arrow and one output arrow. A 32-bit counter display shows 0x00400000 → 0x00400004 → 0x00400008 ticking at each clock edge.

Narration:

> The **Program Counter**. Definition from the notes: the register containing the address of the instruction in the program being executed. It is a **32-bit register**. Its output goes to the instruction memory as the instruction address, and to the adder that computes PC + 4. The notes say the PC is written at the end of every clock cycle, so it does not need a write control signal. Every cycle, it simply loads whatever is at its input — PC + 4, a branch target, or a jump address.

On-screen revision points:

- PC: 32-bit register holding the address of the current instruction.
- Written every clock cycle → no write-control signal.

Scene 3.3 — Instruction Memory

Visual directions: Notes' Figure 2.1(a): box "Instruction memory" with input "Instruction address" and output "Instruction".

Narration:

> The **instruction memory** stores the program. Its input is an instruction address; its output is the 32-bit instruction at that address. The notes point out that the instruction memory only needs to provide **read** access, because the datapath does not write instructions. Since it only reads, we treat it as combinational logic: the output always reflects the contents at the address input, and no read control signal is needed. Why a separate memory for instructions and data? Because in a single-cycle design, no resource can be used more than once per instruction. A load must read an instruction and read data in the same cycle, so we need two memories.

On-screen revision points:

- Instruction memory: read-only in this datapath; no control signal.
- Separate instruction and data memories because single-cycle uses each unit once.

Scene 3.4 — Adders

Visual directions: Notes' Figure 2.1(c): adder shape with two inputs, output "Sum". Then two copies labelled "PC + 4 adder" and "Branch target adder".

Narration:

> An **adder**, in the notes' words, is an ALU wired to always add its two 32-bit inputs and place the sum on its output. Our datapath has **two** adders, apart from the main ALU. The first computes **PC + 4**, the address of the next sequential instruction — one input is the PC, the other is the constant 4. The second computes the **branch target**: PC + 4 plus the shifted, sign-extended offset. Why not reuse the main ALU? Because in a single cycle the ALU is busy — for a beq it is comparing the registers at the same moment.

On-screen revision points:

- Adder #1: PC + 4. Adder #2: branch target = (PC + 4) + (offset << 2).

Scene 3.5 — The Fetch Datapath (Figure 2.2)

Visual directions: Build notes' Figure 2.2 piece by piece: PC → Read address of Instruction memory; PC also → Add with 4; Add output loops back to PC.

Narration:

> Now combine the three elements. The PC sends its value to the instruction memory's read address, and the instruction comes out. At the same time, the PC goes to the adder with the constant 4, and the sum goes back to the PC input. At the next clock edge the PC is updated, and the next instruction is fetched. This small circuit, Figure 2.2 in the notes, is the fetch portion of every instruction. The fetched instruction is then used by other parts of the datapath.

Scene 3.6 — The Register File

Visual directions: Notes' Figure 2.3(a). Register box with five inputs on the left and two outputs on the right; "RegWrite" in blue at the bottom.

Diagram instructions:

```
          5 ─/─► Read register 1          Read data 1 ─/─ 32 ►
          5 ─/─► Read register 2          Read data 2 ─/─ 32 ►
          5 ─/─► Write register
         32 ─/─► Write data
                      RegWrite (control)
```

A slash with a number on a wire means "this many bits".

Narration:

> The **register file**. The notes define it as a state element that consists of a set of registers that can be read and written by supplying a register number to be accessed. It holds all 32 general-purpose registers. R-format instructions have three register operands, so for each instruction we may read two data words and write one. For each word to be **read**, we need an input that specifies the register number, and an output that carries the value. So: **Read register 1**, **Read register 2**, and **Read data 1**, **Read data 2**. To **write**, we need two inputs: the register number — **Write register** — and the value — **Write data**.

> So, four inputs — three register numbers and one data — and two outputs. The register number inputs are **5 bits** wide, since 2 to the power 5 is 32. The data input and the two data outputs are **32 bits** wide. Reads happen all the time: the register file always outputs the contents of whichever registers are on the read inputs. Writes are controlled by the **RegWrite** signal, which must be asserted for a write to happen at the clock edge. In other words, the register file has **two read ports and one write port**.

On-screen revision points:

- Register file: 2 read ports, 1 write port; 4 inputs, 2 outputs.
- Register numbers = 5 bits; data = 32 bits. Write needs RegWrite = 1.

Scene 3.7 — The ALU

Visual directions: Notes' Figure 2.3(b): ALU shape, two 32-bit inputs, "ALU operation" 4-bit control on top, outputs "ALU result" and "Zero".

Narration:

> The **Arithmetic and Logic Unit**, the ALU, operates on the values read from the registers. It has two 32-bit inputs and a 32-bit result. A **4-bit ALU operation** control input selects what it does — add, subtract, AND, OR, or set-on-less-than. It also has a one-bit output called **Zero**, which is 1 when the result is zero. We use Zero to implement beq: subtract the two registers, and if the result is zero, they are equal.

On-screen revision points:

- ALU: two 32-bit inputs, 32-bit result, Zero output, 4-bit ALU operation input.

Scene 3.8 — The Data Memory

Visual directions: Notes' Figure 2.4(a): box "Data memory" with inputs Address, Write data; output Read data; MemWrite on top, MemRead at the bottom (blue).

Narration:

> The **data memory** holds data for loads and stores. It is a state element with two inputs — the **address** and the **write data** — and one output — the **read data**. It has **separate read and write controls**, MemRead and MemWrite, although only one may be asserted on any clock. Why does it need a read signal when the register file does not? The notes explain that, unlike the register file, reading an invalid address in memory can cause problems. So memory is read only when the instruction is a load. Writes are edge-triggered.

On-screen revision points:

- Data memory: inputs Address, Write data; output Read data; MemRead, MemWrite.

Scene 3.9 — The Sign-Extension Unit

Visual directions: Notes' Figure 2.4(b): ellipse "Sign-extend", input 16 bits, output 32 bits. Then two animated bit examples.

Worked example:

```
Positive:  +32 = 0000 0000 0010 0000
   → 0000 0000 0000 0000 0000 0000 0010 0000 = 0x00000020 (= +32)

Negative:  −2  = 1111 1111 1111 1110
   → 1111 1111 1111 1111 1111 1111 1111 1110 = 0xFFFFFFFE (= −2)
```

Narration:

> The **sign-extension unit**. The notes define sign-extend as: to increase the size of a data item by replicating the high-order sign bit of the original data item in the high-order bits of the larger destination data item. The 16-bit immediate or offset must become 32 bits before the ALU or adder can use it. We copy bit 15 — the sign bit — into bits 16 to 31. If the number is positive, bit 15 is 0, so we fill with zeros. +32 stays +32. If it is negative, bit 15 is 1, so we fill with ones. −2 in 16 bits is FFFE, and it becomes FFFF FFFE, which is still −2. This is used by lw, sw, addi, and beq.

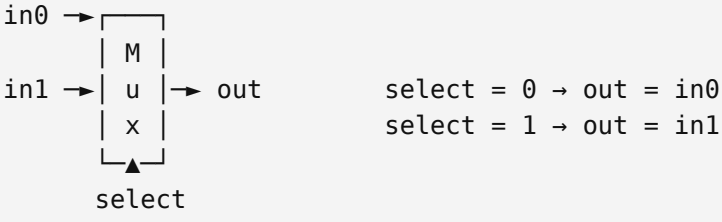
On-screen revision points:

- Sign-extend: copy bit 15 into bits 31–16. Preserves value and sign.

Scene 3.10 — Shift Left 2 and Multiplexors

Visual directions: "Shift left 2" ellipse with wires visibly re-routed; then a 2-input mux with a select line toggling between 0 and 1.

Diagram instructions:



Narration:

> The **shift-left-2 unit** multiplies the branch offset by 4. The notes stress that no actual shift hardware is needed, since the amount is constant. It is just wiring that adds 0 0 at the low end. The shift throws away the top two bits, which are only sign bits after sign extension. A **multiplexor**, or **data selector**, chooses one of its inputs and passes it to the output, based on a select control line. With two inputs, one select bit is enough: 0 picks the first input, 1 picks the second. The complete datapath has these multiplexors:

> **ALUSrc mux** — ALU second input: register or sign-extended immediate. **MemtoReg mux** — register write data: ALU result or memory data. **PCSrc mux** — next PC: PC + 4 or branch target. And, as a clarification beyond the notes' figure, a **RegDst mux** choosing rt or rd as the write register, and a **Jump mux** for the jump address.

On-screen revision points:

- Multiplexor = data selector; select line picks the input.
- Muxes: ALUSrc, MemtoReg, PCSrc (+ RegDst, Jump as clarifications).

Scene 3.11 — The Control Unit

Visual directions: "Control" ellipse with the 6-bit opcode (instruction bits 31–26) as input and blue signal wires fanning out to every unit.

Narration:

> The **control unit** has the instruction as its input. It looks at the opcode — bits 31 to 26 — and sets the control lines for the functional units and the multiplexors. For R-type instructions the opcode is 0, so the ALU operation is further determined by the **funct** field, bits 5 to 0. One multiplexor is not set directly by the control unit: the PCSrc mux. It is set by ANDing the **Branch** control signal with the ALU's **Zero** output. Think of the control unit as a conductor of an orchestra. The datapath components are the musicians; they can all play, but the control unit decides who plays what on each instruction.

On-screen revision points:

- Control unit input: opcode (bits 31–26). Outputs: RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp (+ Jump).
- PCSrc = Branch AND Zero.

Scene 3.12 — The Complete Datapath (Figure 4.11)

Visual directions: The complete datapath assembles itself from the components introduced, one at a time, then the full labelled diagram holds on screen for 10 seconds.

Diagram instructions (notes' Figure 4.11, with clarifications in dashed lines):

- **PC** → Read address of **Instruction memory**; PC → **Add** (with **4**).
- Add output (PC + 4) → **Add** (branch adder, labelled "ALU result") and → input 0 of **PCSrc Mux** (top right). Branch adder output → input 1 of PCSrc Mux. PCSrc Mux output → back to PC.
- Instruction [25:21] → **Read register 1**; [20:16] → **Read register 2**; [20:16] → **Write register** (dashed **RegDst mux** with [15:11] as second input — clarification); [15:0] → **Sign-extend** (16 → 32).
- Sign-extend output → **Shift left 2** → branch adder; and → input 1 of **ALUSrc Mux**.
- **Read data 1** → ALU top input. **Read data 2** → ALUSrc Mux input 0 and → **Write data** of Data memory.

- **ALU** (control "ALU operation", 4 bits) → **ALU result** → Data memory **Address** and → MemtoReg Mux input 0. ALU **Zero** → branch logic.
- **Data memory** (MemWrite top, MemRead bottom) Read data → **MemtoReg Mux** input 1.
- MemtoReg Mux output → loops along the bottom to the register file **Write data**.
- Control labels in blue: PCSrc, ALUSrc, ALU operation, MemWrite, MemtoReg, MemRead, RegWrite.
- Caption (from notes): "The simple datapath for the core MIPS architecture combines the elements required by different instruction classes."

Narration:

> Here it is — the complete single-cycle datapath, Figure 4.11 in the notes. Let us trace it from left to right. The **PC** feeds the **instruction memory** and the **PC + 4 adder**. The instruction is split into fields. Bits 25 to 21 go to read register 1. Bits 20 to 16 go to read register 2 and to the write register input. Bits 15 to 0 go to the sign-extend unit. Read data 1 goes straight into the ALU. Read data 2 goes to the **ALUSrc multiplexor** and also to the data memory's write data input — that is the path used by store word. The sign-extended immediate goes into the ALUSrc multiplexor, and also through **shift left 2** into the **branch adder**. The ALU result goes to the data memory address and to the **MemtoReg multiplexor**. The memory read data also goes to MemtoReg. Its output loops back to the register file's write data. At the top, the **PCSrc multiplexor** chooses between PC + 4 and the branch target, and feeds the PC. The notes say this datapath can execute the basic instructions — load-store word, ALU operations and branches — in a single clock cycle. Just one additional multiplexor is needed to integrate branches. Support for jumps is added later. A clarification: in the notes' figure, the write register is wired only from bits 20 to 16 — that works for lw, but an R-type instruction writes rd, bits 15 to 11. The full textbook design adds a **RegDst multiplexor** here, shown dashed.

Scene 3.13 — Hardware Path: R-Format, LW, SW, BEQ on the Complete Datapath

Visual directions: Four consecutive animations on the same full datapath, each in its own colour. Unused units fade to 20% opacity. A side panel lists the active control signals.

Narration:

> Let us run all four instruction types through the complete hardware. **R-format, `add $t1,$t2,$t3`, in green.** PC → instruction memory. rs and rt are read. ALUSrc = 0, so read data 2 enters the ALU. The ALU adds. Data memory is idle. MemtoReg = 0, so the ALU result travels back to write data. RegWrite = 1. PCSrc = 0, so PC + 4 is loaded. **LW, `lw $t0,32($s3)`, in blue.** rs is read. The offset is sign-extended, and ALUSrc = 1 sends it into the ALU. The ALU adds to form the address. MemRead = 1. Memory data goes through MemtoReg = 1 to write data. RegWrite = 1. PC + 4. **SW, `sw $t0,48($s3)`, in purple.** rs and rt are read. ALUSrc = 1; the ALU forms the address. Read data 2 flows along the lower wire into memory's write data. MemWrite =

1. RegWrite = 0. PC + 4. **BEQ, `beq $t1,$t2,L`, in orange.** rs and rt are read. ALUSrc = 0; the ALU subtracts. Meanwhile the offset is sign-extended, shifted left 2 and added to PC + 4. If Zero = 1, PCSrc = 1 and the target is loaded into the PC; otherwise PC + 4. RegWrite = 0, MemWrite = 0. The same hardware, four different behaviours — selected entirely by control signals.

On-screen revision points: | Component | Purpose | Used by | |---|---|---| | PC | address of current instruction | all | | Instruction memory | supplies instruction | all | | Adder (PC + 4) | next sequential address | all | | Register file | read rs, rt; write rd/rt | R, lw, sw, beq | | Sign-extend | 16 → 32 bits | lw, sw, beq, addi | | ALUSrc mux | register vs immediate | all with ALU | | ALU | operation / address / compare | R, lw, sw, beq | | Data memory | load / store data | lw, sw | | MemtoReg mux | ALU vs memory to register | R, lw | | Shift left 2 + adder | branch target | beq | | PCSrc mux | PC + 4 vs target | all | | Control unit | sets all control lines | all |

Exam points for Topic 3:

- Draw the complete datapath and label every unit and control line.
- Register file: 2 read ports, 1 write port; 5-bit register numbers.
- Instruction memory needs no read control; data memory needs MemRead/MemWrite.
- Shift left 2 is just wiring.

End of Part 3 — the three priority topics are complete. Continue with Part 4.

PART 4 — Topics 4 to 7: Eight Great Ideas, Functional Units, Basic Operational Concepts, CPU Performance

---

Scene 4.0 — Computer Architecture: Subcategories, CISC and RISC

Visual directions: Heading "Before the Eight Great Ideas — from the opening of the notes". Three boxes (ISA, Microarchitecture, Systems design), then a CISC vs RISC two-column comparison.

Narration:

> The notes open Unit 3 by saying computer architecture has three main subcategories. **Instruction set architecture, or ISA**: the functions and capabilities of the central processing unit — the programming language it understands, its data formats and the instruction set. MIPS is an ISA. **Microarchitecture**, also known as computer organization: how a particular processor implements the ISA. The datapath we just built is a microarchitecture. **Systems design**: all the hardware components of a computer — data processors, multiprocessors, memory controllers and so on. The notes also describe two approaches to processor architecture. **CISC**, Complex Instruction Set Computer: a small register set and hundreds of different instructions, with a single instruction for a task.

> Fewer lines of code and less memory, but instructions can take longer. **RISC**, Reduced Instruction Set Computer: a smaller set of instructions with few addressing modes; instructions execute faster and take one clock cycle per instruction. MIPS is a RISC — which is why it has only five addressing modes and three formats.

On-screen revision points: | CISC | RISC | |---|---| | Hundreds of instructions, small register set | Small instruction set, few addressing modes | | Fewer lines of code, less memory | Faster execution, one clock cycle per instruction |

Scene 4.1 — Topic 4: Eight Great Ideas in Computer Architecture (overview)

Visual directions: Heading "TOPIC 4 — EIGHT GREAT IDEAS". Eight icon tiles appear in a 2×4 grid: rocket (Moore's Law), layered cake (Abstraction), sports car (Common case fast), parallel arrows (Parallelism), pipeline (Pipelining), crystal ball (Prediction), pyramid (Hierarchy), spare tyre (Redundancy).

Narration:

> The notes say these are eight great ideas that computer architects have invented in the last 60 years of computer design. Here they are: Design for Moore's Law. Use abstraction to simplify design. Make the common case fast. Performance via parallelism. Performance via pipelining. Performance via prediction. Hierarchy of memories. Dependability via redundancy. Let us take them one by one.

Scene 4.2 — Idea 1: Design for Moore's Law

Visual directions: Graph of transistor count doubling every two years; a project timeline bar labelled "design starts" and "design finishes" with an arrow pointing to the future.

Narration:

> **Definition:** Moore's Law says the number of transistors in an integrated circuit doubles approximately every two years. It is named after Gordon Moore, one of the founders of Intel. **Significance:** computer designs can take years. During that time, the resources available per chip can easily double or quadruple between the start and the finish of the project. So, as the notes say, architects must anticipate where the technology will be **when the design finishes**, rather than design for where it starts. **Example:** a team starting a 4-year chip project designs for about four times today's transistor budget, because two doublings will happen before release.

Scene 4.3 — Idea 2: Use Abstraction to Simplify Design

Visual directions: Stack of layers; each layer hides the one below.

Narration:

> **Definition:** an abstraction level is a generalization of a model or algorithm. Lower-level details are hidden to offer a simpler model at higher levels. The notes say a computer system is usually represented as five abstraction levels: hardware, firmware, assembler, operating system and processes. **Significance:** both architects and programmers needed techniques to be more productive; otherwise design time would lengthen dramatically as resources grew with Moore's Law. A good abstraction layer also allows easy reuse. **Example:** a C programmer writes `f = g + h` without thinking about registers; the compiler turns it into `add`; the hardware designer builds an ALU without knowing which program will run. We used abstraction in Topic 3 too — we drew the register file as one box, hiding thousands of gates.

Scene 4.4 — Idea 3: Make the Common Case Fast

Visual directions: Pie chart: 90% "common operations", 10% "rare". A speed-up arrow on the 90% slice.

Narration:

> **Definition:** making the common case fast tends to enhance performance better than optimizing the rare case. **Significance:** it implies you know what the common case is, which the notes say is only possible with careful experimentation and measurement. **Example:** in MIPS, most instructions use registers, so registers are made very fast and plentiful. Most constants are small, so a 16-bit immediate field is enough for the common case.

Scene 4.5 — Idea 4: Performance via Parallelism

Visual directions: One large job split into four pieces processed by four workers simultaneously.

Narration:

> **Definition:** computer architects offer designs that get more performance by performing operations in **parallel**. Parallel computing is a form of computation in which many calculations are carried out simultaneously, based on the principle that large problems can often be divided into smaller ones, which are then solved concurrently. **Significance:** the notes say parallelism has been used for many years, mainly in high-performance computing. **Example:** adding two arrays of a million numbers on four cores — each core adds a quarter, so ideally it finishes in a quarter of the time. Even in our datapath, the ALU comparison and the branch-target adder work in parallel during a beq.

Scene 4.6 — Idea 5: Performance via Pipelining

Visual directions: Laundry analogy (wash, dry, fold) then an instruction pipeline diagram: 5 instructions × 5 stages, staggered.

Diagram instructions:

```
Time →     1    2    3    4    5    6    7
Instr 1:  IF   ID   EX   MEM  WB
Instr 2:       IF   ID   EX   MEM  WB
Instr 3:            IF   ID   EX   MEM  WB
```

Narration:

> **Definition from the notes:** pipelining is a technique used to increase the instruction **throughput** — the number of instructions that can be executed in a unit of time. The basic instruction cycle is broken into a series of pipeline stages. Instead of finishing one instruction before starting the next, each instruction is split into steps, and different steps of different instructions run at the same time on different circuitry. **Key exam point:** pipelining increases throughput but does **not** reduce instruction **latency** — the time to complete a single instruction — because each instruction must still go through all the steps. **Analogy:** in a laundry, while load 1 is drying, load 2 is washing. Each load still takes the same time, but more loads finish per hour.

Scene 4.7 — Idea 6: Performance via Prediction

Visual directions: A pipeline reaches a branch "fork in the road". A crystal ball guesses "taken"; instructions are fetched along the guessed path.

Narration:

> **Definition:** to improve flow and throughput in an instruction pipeline, **branch predictors** guess the outcome of a branch. **Significance:** without branch prediction, the processor would have to wait until the conditional jump has passed the execute stage before the next instruction could enter the fetch stage. The predictor avoids this waste of time by guessing whether the branch is most likely to be taken or not taken. **Example:** in a loop that runs 100 times, the branch back to the top is taken 99 times. Predicting "taken" is right 99% of the time.

Scene 4.8 — Idea 7: Hierarchy of Memories

Visual directions: Reproduce the notes' memory pyramid: CPU Register at the top; Cache (Level 1, Level 2); RAM (Physical RAM, Virtual Memory) — bracket "Temporary Storage Areas"; Storage Device Types (ROM/BIOS, Removable drives, Network/Internet storage, Hard drive) — bracket "Permanent Storage Areas"; Input Sources (Keyboard, Mouse, Removable media, Scanner/Camera/Mic/Video, Remote source, Other sources) at the base. Side arrows: "faster, smaller, costlier per bit ↑" and "slower, larger, cheaper per bit ↓".

Narration:

> **Definition:** programmers want memory to be fast, large and cheap. Speed shapes performance, capacity limits the problem size, and memory cost is often the majority of computer cost. These demands conflict. Architects solve this with a **hierarchy of memories**: the fastest, smallest and most expensive memory per bit at the top, and the

> slowest, largest and cheapest per bit at the bottom. **Significance:** caches give the programmer the illusion that main memory is nearly as fast as the top of the hierarchy, and nearly as big and cheap as the bottom. **Example:** registers → cache → RAM → hard disk.

Scene 4.9 — Idea 8: Dependability via Redundancy

Visual directions: Two servers, one fails (red X), the other takes over (green check). Label "failover / high availability".

Narration:

> **Definition:** computers need to be not only fast but **dependable**. Since any physical device can fail, we include **redundant** components that can take over when a failure occurs and help detect failures. **Example from the notes:** system designers provide **failover** capability in servers, systems or networks requiring continuous availability. This is called **high availability** — a high degree of reliability. Another everyday example: RAID disk arrays keep duplicate copies of data.

On-screen revision points (Topic 4): | # | Idea | One-line meaning | |---|---|---| | 1 | Design for Moore's Law | Transistors double ~2 years; design for the finish date | | 2 | Abstraction | Hide lower-level detail; 5 levels | | 3 | Common case fast | Optimize the frequent case (measure it) | | 4 | Parallelism | Many operations at the same time | | 5 | Pipelining | Overlap stages; ↑ throughput, not ↓ latency | | 6 | Prediction | Guess branch outcome instead of waiting | | 7 | Hierarchy of memories | Fast/small/costly on top, slow/large/cheap below | | 8 | Redundancy | Spare components → failover, high availability |

Scene 5.1 — Topic 5: Functional Units of a Computer System

Visual directions: Heading "TOPIC 5 — FUNCTIONAL UNITS". Reproduce the notes' block diagram: left box "I/O" containing "Input" and "Output"; centre box "Memory"; right box "Processor" containing "ALU" and "Control Unit". Then animated arrows show information flow.

Diagram instructions:



Narration:

> The notes say a computer consists of five functionally independent main parts: **Input**, **Memory**, **Arithmetic and Logic Unit**, **Output** and **Control unit**. Here is how information flows. The input device accepts coded information — a source program in a high-level language, or data. This is either stored in memory or used immediately by the processor. The program stored in memory determines the processing steps. Basically, the

> computer converts a source program into an object program — that is, machine language. Finally, the results go to the outside world through an output device. All of these actions are coordinated by the control unit.

Scene 5.2 — Input Unit and Output Unit

Visual directions: Keyboard key pressed → binary code travels along a cable → memory/processor. Then printer, speaker and monitor icons for output.

Narration:

> **Input unit:** the program or data is fed into the computer through input devices. The keyboard is the most common. Whenever a key is pressed, the corresponding word or number is translated into its equivalent binary code over a cable and fed either to memory or to the processor. Other input devices: joysticks, trackballs, mouse, scanners. **Output unit:** the counterpart of the input unit. Its basic function is to send the processed results to the outside world. Examples: printer, speakers, monitor.

Scene 5.3 — Memory Unit: Primary Memory

Visual directions: A grid of memory cells grouped into words, each word with an address on the left (0, 1, 2 …). Labels for "word length", "address", "RAM", "ROM", "Cache".

Narration:

> **Memory unit:** its function is to store programs and data. There are two types: primary memory and secondary memory. **Primary memory** is the one exclusively associated with the processor and operates at electronic speeds. Programs must be stored in this memory while they are being executed. It contains a large number of semiconductor storage cells, each capable of storing one bit. These are processed in groups of fixed size called **words**. To provide easy access to a word, a distinct **address** is associated with each word location. Addresses are numbers that identify memory locations. The number of bits in each word is called the **word length** of the computer. Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called **random-access memory, RAM**. The time required to access one word is called the **memory access time**. Memory that is only readable by the user, whose contents cannot be altered, is called **read-only memory, ROM**. The notes say it contains the operating system. **Caches** are small, fast RAM units coupled with the processor, often on the same IC chip, to achieve high performance. Primary storage is essential, but it tends to be expensive.

On-screen revision points:

- Word: fixed-size group of bits. Word length: bits per word.
- Address: number identifying a memory location.
- RAM: any location reachable in short, fixed time. Access time: time to access one word.
- ROM: read-only. Cache: small fast RAM near/on the processor.

Scene 5.4 — Memory Unit: Secondary Memory

Visual directions: Icons: magnetic disk, tape, CD-ROM, floppy.

Narration:

> **Secondary memory** is used where large amounts of data and programs must be stored, particularly information that is accessed infrequently. Examples: magnetic disks and tapes, optical disks such as CD-ROMs, and floppies. It is cheaper and larger than primary memory, but slower.

On-screen revision points: | Primary | Secondary | |---|---| | Directly used by processor, electronic speed | Large, infrequently accessed data | | RAM, ROM, cache | Magnetic disks/tapes, CD-ROM, floppy | | Expensive per bit | Cheap per bit |

Scene 5.5 — ALU and Control Unit

Visual directions: Operands travel memory → registers → ALU → result. Then the control unit drawn as a "nerve centre" with timing pulses going to each unit.

Narration:

> **Arithmetic and logic unit:** most computer operations — addition, subtraction, multiplication, division and so on — are executed in the ALU. Operands are brought into the ALU from memory and stored in high-speed storage elements called **registers**. Then the operation is performed according to the instructions. The control unit and the ALU are many times faster than other devices connected to the computer. This lets a single processor control many external devices — keyboards, displays, magnetic and optical disks, sensors and mechanical controllers. **Control unit:** it is effectively the **nerve centre** that sends signals to other units and senses their states. The actual **timing signals** that govern the transfer of data between the input unit, processor, memory and output unit are generated by the control unit.

On-screen revision points:

- 5 units: Input, Memory, ALU, Output, Control. ALU + Control = Processor.
- Control unit = nerve centre; generates timing signals.

Scene 6.1 — Topic 6: Basic Operational Concepts — Stored Program

Visual directions: Heading "TOPIC 6 — BASIC OPERATIONAL CONCEPTS". Memory shows a list of instructions and some data words.

Narration:

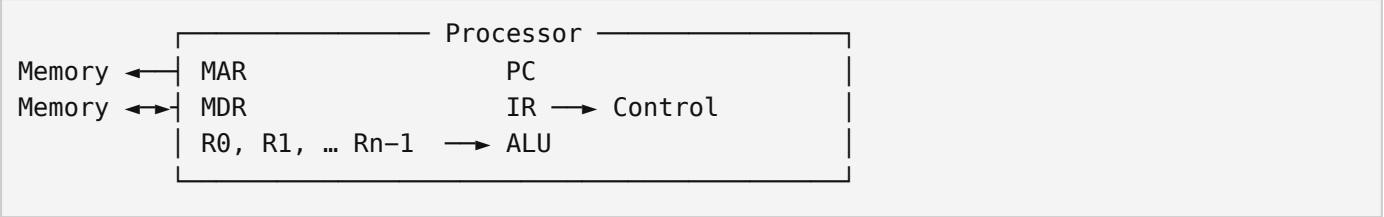
> To perform a task, an appropriate program — a list of instructions — is stored in memory. Individual instructions are brought from memory into the processor, which executes them.

> Data are also stored in memory. The notes' example is `Add LOCA, R0`. This instruction adds the operand at memory location LOCA to the operand in register R0 and places the sum into register R0. It needs several steps: One — the instruction is fetched from memory into the processor. Two — the operand at LOCA is fetched and added to the contents of R0. Three — the resulting sum is stored in register R0. This instruction combines a memory access with an ALU operation. In some other computers — like MIPS, which is a load-store machine — these are done by separate instructions for performance reasons: `Load LOCA, R1` then `Add R1, R0`. Note: this example uses a generic textbook notation, not MIPS syntax. Here, the destination is written last.

Scene 6.2 — Processor Registers: PC, IR, MAR, MDR

Visual directions: Processor block diagram: Memory on the left connected via two lines to MAR and MDR inside the processor; inside also PC, IR, Control, ALU, and general-purpose registers R0 … Rn−1.

Diagram instructions:



Narration:

> Transfers between memory and the processor start by sending the address of the location to the memory unit and issuing the appropriate control signals. Then the data are transferred to or from memory. Four special registers are involved: **PC — Program Counter**: holds the address of the next instruction to be fetched. **IR — Instruction Register**: holds the instruction currently being executed. **MAR — Memory Address Register**: holds the address of the location to be accessed. **MDR — Memory Data Register**: contains the data to be written into, or read out of, the addressed location.

On-screen revision points:

- MAR: address of location to access. MDR: data read from / written to memory.
- PC: address of next instruction. IR: current instruction.

Scene 6.3 — The 12 Operating Steps, Animated with ADD LOCA, R0

Visual directions: The block diagram from 6.2 with values flowing. Each numbered step appears on the right as it happens.

Worked example (values added for illustration):

```
Memory: address 100: "Add LOCA, R0"   LOCA = 500, Memory[500] = 25
R0 = 10 before execution
```

Narration (the notes' operating steps, with the example):

> 1. Programs reside in memory, usually arriving through the input unit.
> 2. Execution starts when the PC is set to point at the first instruction. PC = 100.
> 3. The contents of the PC are transferred to the MAR, and a **Read** control signal is sent to memory. MAR = 100.
> 4. After the memory access time elapses, the addressed word is read out of memory and loaded into the MDR. MDR = "Add LOCA, R0".
> 5. The contents of MDR are transferred to the IR. Now the instruction is ready to be decoded and executed.
> 6. The instruction involves an ALU operation, so the required operands must be obtained. R0 is already in the processor: 10.
> 7. The operand in memory is fetched by sending its address to MAR and initiating a read cycle. MAR = 500.
> 8. When the operand has been read into the MDR, it is transferred from MDR to the ALU. MDR = 25.
> 9. After one or two such cycles, the ALU performs the operation: 25 + 10 = 35.
> 10. If the result were to be stored in memory, it would be sent to the MDR.
> 11. And the address of that location would be sent to MAR and a write cycle started. In our example, the result goes into R0 instead, so R0 = 35.
> 12. The contents of the PC are incremented to point to the next instruction. PC = 104 for a 4-byte instruction.

On-screen revision points:

- Fetch: PC → MAR → Read → MDR → IR.
- Operand fetch: address → MAR → Read → MDR → ALU.
- Store: result → MDR, address → MAR → Write.
- PC incremented to next instruction.

Scene 6.4 — Interrupts and Interrupt Service Routines

Visual directions: A program running; a printer icon raises a red "INT" flag; the processor saves its state into memory, jumps to an ISR, returns, restores state.

Narration:

> Normal execution of a program may be **preempted** — temporarily interrupted — if some device needs urgent servicing. To do this, the device raises an **interrupt** signal. An **interrupt** is a request signal from an I/O device for service by the processor. The processor provides the service by executing an appropriate **interrupt service routine**, ISR. Because this diversion may change the internal state of the processor, its state must be saved in memory before servicing the interrupt. When the ISR is completed, the state is restored so the interrupted program can continue.

On-screen revision points:

- Interrupt: request from an I/O device for service.
- ISR: routine that services it. Save state → run ISR → restore state → resume.

Scene 7.1 — Topic 7: Performance — Response Time and Throughput

Visual directions: Heading "TOPIC 7 — CPU PERFORMANCE". Split screen: one user at a desktop with a stopwatch ("response time"); a datacenter with a counter of jobs per day ("throughput").

Narration:

> What does "faster" mean? The notes give two views. If you run a program on two desktop computers, the faster one gets the job done first. As an individual user, you care about **response time** — the time between the start and completion of a task, also called **execution time**. A datacenter manager running many users' jobs calls the faster computer the one that completes the most jobs in a day. They care about **throughput**, or **bandwidth** — the total amount of work done in a given time.

Scene 7.2 — Performance = 1 / Execution Time

Visual directions: Formula builds step by step.

On-screen formulas:

```
Performance_X = 1 / Execution time_X

Performance_X > Performance_Y
⇔ 1/Execution time_X > 1/Execution time_Y
⇔ Execution time_Y > Execution time_X

"X is n times as fast as Y":
Performance_X / Performance_Y = Execution time_Y / Execution time_X = n
```

Narration:

> To maximize performance, we minimize execution time. So performance is the reciprocal of execution time. If X performs better than Y, then 1 over X's time is larger than 1 over Y's time, so Y's execution time is longer. "X is n times as fast as Y" means performance of X divided by performance of Y equals n — which equals execution time of **Y** divided by execution time of **X**. Notice the swap: the slower machine's time goes on top.

Scene 7.3 — Measuring Performance: CPU Time, User and System Time

Visual directions: A timeline bar for one program: segments "user CPU", "system CPU", "waiting for I/O", "other programs". Brackets show which count as CPU time.

Narration:

> **CPU execution time**, or **CPU time**: the actual time the CPU spends computing for a specific task. It does not count I/O waiting or time spent running other programs. It splits into: **User CPU time** — CPU time spent in the program itself. **System CPU time** — CPU time spent in the operating system performing tasks on behalf of the program.

Worked example (beyond the notes, for practice):

```
User CPU time = 90 s, System CPU time = 12 s, elapsed (wall-clock) time = 159 s
CPU time = 90 + 12 = 102 s
Fraction of elapsed time on CPU = 102 / 159 = 0.6415 ≈ 64%
The remaining 57 s (36%) was I/O or other programs.
```

Scene 7.4 — The CPU Performance Equations

Visual directions: Each formula appears; every variable is underlined and labelled.

On-screen formulas:

```
(1) CPU time = CPU clock cycles × Clock cycle time
(2) CPU time = CPU clock cycles / Clock rate          (Clock rate = 1 / Clock cycle
time)
(3) CPU clock cycles = Instruction count × CPI
(4) CPU time = Instruction count × CPI × Clock cycle time
(5) CPU time = (Instruction count × CPI) / Clock rate

seconds/program = instructions/program × cycles/instruction × seconds/cycle
```

Narration:

> The bottom-line measure is CPU execution time. The processor runs on a clock. One **clock cycle** is one tick; **clock cycle time** is its duration, in seconds — for example 250 picoseconds. **Clock rate** is ticks per second, in hertz — for example 4 gigahertz. They are inverses: 1 over 250 picoseconds is 4 gigahertz. So CPU time equals the number of clock cycles times the clock cycle time — or the number of clock cycles divided by the clock rate. How many clock cycles? The program executes a number of instructions — the **instruction count**, IC. Each takes some clock cycles; the average is **CPI**, **clock cycles per instruction**. So CPU clock cycles equal IC times CPI. Putting it together — the classic performance equation: **CPU time = Instruction count × CPI × Clock cycle time.** Check the units: instructions per program, times cycles per instruction, times seconds per cycle — instructions and cycles cancel, giving seconds per program. The notes say CPI is an average over all instructions executed, because different instructions take different

> times. And CPI is a good way to compare two implementations of the same ISA, because their instruction counts are the same.

On-screen revision points — what affects each factor (notes' table): | Component | Affects | |---|---| | Algorithm | Instruction count, possibly CPI | | Programming language | Instruction count, CPI | | Compiler | Instruction count, CPI | | Instruction set architecture | Instruction count, clock rate, CPI |

Unit reminders: 1 GHz = 10^9 Hz; 1 ns = 10^{-9} s; 1 ps = 10^{-12} s.

Scene 7.5 — Problem 1 (from the notes): How Much Faster?

Visual directions: Problem text, then calculation lines appear one at a time.

Worked example:

```
Computer A runs a program in 10 s; computer B runs it in 15 s. How much faster is A?

Performance_A / Performance_B = Execution time_B / Execution time_A = n
n = 15 s / 10 s = 1.5          (seconds cancel → n has no unit)
Answer: A is 1.5 times as fast as B.
```

Narration:

> Performance A over performance B equals time B over time A. 15 seconds over 10 seconds is 1.5. The seconds cancel, so the answer is a pure ratio. A is 1.5 times as fast as B.

Scene 7.6 — Simple Practice: Using CPU Time = IC × CPI × Cycle Time

Worked example (beyond the notes):

```
A program executes 2 × 10⁶ instructions, average CPI = 2, clock rate = 1 GHz.

Step 1: Clock cycle time = 1 / clock rate = 1 / (1 × 10⁹ Hz) = 1 × 10⁻⁹ s = 1 ns
Step 2: CPU clock cycles = IC × CPI = 2 × 10⁶ × 2 = 4 × 10⁶ cycles
Step 3: CPU time = cycles × cycle time = 4 × 10⁶ × 1 × 10⁻⁹ s = 4 × 10⁻³ s = 4 ms
Check:  CPU time = cycles / clock rate = 4 × 10⁶ / 10⁹ = 0.004 s ✔
```

Narration:

> First the cycle time: one over one gigahertz is one nanosecond. Then cycles: two million instructions times two cycles each is four million cycles. Then time: four million times one nanosecond is four milliseconds. We check with the clock-rate form and get 0.004 seconds — the same answer.

Scene 7.7 — Problem 2 (from the notes): Target Clock Rate

Worked example:

```
Program runs 10 s on computer A with a 2 GHz clock. Designer wants computer B to run
it in 6 s, but B will need 1.2 × as many clock cycles as A. What clock rate for B?

Given: Clock rate_A = 2 GHz = 2 × 10⁹ cycles/s; CPU time_A = 10 s;
       CPU time_B = 6 s; CPU clock cycles_B = 1.2 × CPU clock cycles_A

Step 1: CPU time_A = CPU clock cycles_A / Clock rate_A
        10 s = CPU clock cycles_A / (2 × 10⁹ cycles/s)
        CPU clock cycles_A = 10 s × 2 × 10⁹ cycles/s = 20 × 10⁹ cycles

Step 2: CPU time_B = 1.2 × CPU clock cycles_A / Clock rate_B
        6 s = 1.2 × 20 × 10⁹ cycles / Clock rate_B

Step 3: Clock rate_B = (1.2 × 20 × 10⁹ cycles) / 6 s
                     = 24 × 10⁹ / 6 = 4 × 10⁹ cycles/s = 4 GHz

Answer: B needs 4 GHz — twice the clock rate of A.
```

Narration:

> First find A's cycles: time times rate, 10 seconds times 2 times 10 to the 9 cycles per second gives 20 times 10 to the 9 cycles. B needs 1.2 times that: 24 times 10 to the 9 cycles. To do it in 6 seconds, B's rate is 24 times 10 to the 9 divided by 6, which is 4 times 10 to the 9 cycles per second — 4 gigahertz. Twice A's clock rate.

Scene 7.8 — Problem 3 (from the notes): Same ISA, Different CPI and Cycle Time

Worked example:

```
A: cycle time 250 ps, CPI 2.0.  B: cycle time 500 ps, CPI 1.2. Same ISA → same IC = I.

Step 1: CPU clock cycles_A = I × 2.0 ; CPU clock cycles_B = I × 1.2
Step 2: CPU time_A = I × 2.0 × 250 ps = 500 × I ps
        CPU time_B = I × 1.2 × 500 ps = 600 × I ps
Step 3: A is faster (smaller time).
        Performance_A / Performance_B = 600 I ps / 500 I ps = 1.2
Answer: A is 1.2 times as fast as B for this program.
```

Narration:

> We do not know the instruction count, but it is the same, so we call it I and it cancels at the end. A: I times 2 times 250 picoseconds is 500 I picoseconds. B: I times 1.2 times 500 is 600 I picoseconds. Ratio: 600 over 500 is 1.2. A has the worse CPI but the much faster clock, and wins by a factor of 1.2.

Scene 7.9 — Problem 4 (from the notes): Comparing Code Sequences — fully solved

Visual directions: Tag "Notes solve only part (i); parts (ii) and (iii) completed here. Notes print 'Code sequence 1' twice — the second line refers to sequence 2."

Worked example:

```
CPI per class:        A = 1, B = 2, C = 3
Instruction counts:   Seq 1: A=2, B=1, C=2      Seq 2: A=4, B=1, C=1

(i) Instruction count
    Seq 1 = 2 + 1 + 2 = 5 instructions
    Seq 2 = 4 + 1 + 1 = 6 instructions
    → Sequence 2 executes more instructions.

(ii) CPU clock cycles = Σ (CPI_i × C_i)
    Seq 1 = (2×1) + (1×2) + (2×3) = 2 + 2 + 6 = 10 cycles
    Seq 2 = (4×1) + (1×2) + (1×3) = 4 + 2 + 3 = 9 cycles
    → Sequence 2 is faster (9 < 10), even though it executes more instructions.

(iii) CPI = CPU clock cycles / Instruction count
    CPI_1 = 10 / 5 = 2.0
    CPI_2 = 9 / 6  = 1.5
```

Narration:

> Part one: sequence 1 has 5 instructions and sequence 2 has 6. So sequence 2 executes more instructions. Part two: count cycles by multiplying each class's count by its CPI and adding. Sequence 1: 2 times 1, plus 1 times 2, plus 2 times 3, is 10 cycles. Sequence 2: 4 plus 2 plus 3 is 9 cycles. Sequence 2 is faster. Part three: CPI is cycles divided by instructions. 10 over 5 is 2.0; 9 over 6 is 1.5. The lesson: **instruction count alone does not decide performance** — you must use all three factors.

Scene 7.10 — Exam-Level Problem: Two Compilers

Worked example (beyond the notes):

```
A 2 GHz machine. CPI: class A = 1, B = 2, C = 3.
Compiler 1: A = 5 × 10⁹, B = 1 × 10⁹, C = 1 × 10⁹ instructions
Compiler 2: A = 10 × 10⁹, B = 1 × 10⁹, C = 1 × 10⁹ instructions

IC_1 = 7 × 10⁹ ; IC_2 = 12 × 10⁹
Cycles_1 = 5×1 + 1×2 + 1×3 = 10 × 10⁹ cycles
Cycles_2 = 10×1 + 1×2 + 1×3 = 15 × 10⁹ cycles
CPU time_1 = 10 × 10⁹ / 2 × 10⁹ Hz = 5 s
CPU time_2 = 15 × 10⁹ / 2 × 10⁹ Hz = 7.5 s
CPI_1 = 10/7 = 1.43 ; CPI_2 = 15/12 = 1.25
Compiler 1 is faster by 7.5 / 5 = 1.5 times — even though compiler 2 has the lower CPI.
```

Narration:

> This is the trap examiners like: compiler 2 has a better CPI, 1.25 against 1.43, but it produces many more instructions. Total time is what matters — compiler 1 takes 5 seconds, compiler 2 takes 7.5 seconds. Compiler 1 is 1.5 times as fast. Never judge performance by CPI alone or by clock rate alone.

On-screen revision points (Topic 7):

- Performance = 1 / Execution time; n = Time_slow / Time_fast.
- CPU time = IC × CPI × Clock cycle time = IC × CPI / Clock rate.
- CPU clock cycles = IC × CPI = Σ $CPI_i \times C_i$.
- CPU time = user CPU time + system CPU time (excludes I/O and other programs).

End of Part 4. Continue with Part 5.

PART 5 — Topics 8 to 10 and the Final Revision

Scene 8.1 — Topic 8: Instructions — Language of the Computer

Visual directions: Heading "TOPIC 8 — OPERATIONS AND OPERANDS". A dictionary icon labelled "Instruction set" with words inside labelled "instructions".

Narration:

> The notes put it simply: the words of a computer's language are called **instructions**, and its vocabulary is called an **instruction set**. Every instruction has an **operation** — what to do — and **operands** — what to do it on. MIPS has three kinds of operands: register operands, memory operands, and constant or immediate operands. Then we look at four kinds of operations: arithmetic, data transfer, logical, and conditional.

Scene 8.2 — Register Operands

Visual directions: The 32-register file with assembler names. Then the C-to-MIPS example from the notes animated with values.

Narration:

> **Register operands.** Arithmetic instructions use register operands. MIPS has a **32 × 32-bit register file** — 32 registers, each 32 bits. They are used for frequently accessed data, numbered 0 to 31. 32-bit data is called a **word**. Assembler names: $t0 to $t9 for **temporary** values, and $s0 to $s7 for **saved** variables. The notes' example: the C statement f = (g + h) − (i + j), with f, g, h, i, j in $s0 to $s4.

Worked example:

```
f = (g + h) - (i + j);    f→$s0, g→$s1, h→$s2, i→$s3, j→$s4
Given g = 10, h = 20, i = 3, j = 4

add $t0, $s1, $s2   # $t0 = g + h = 10 + 20 = 30    dest $t0, sources $s1,$s2
add $t1, $s3, $s4   # $t1 = i + j = 3 + 4  = 7      dest $t1, sources $s3,$s4
sub $s0, $t0, $t1   # f   = 30 − 7          = 23     dest $s0, sources $t0,$t1
```

Narration (continued):

> The compiler uses two temporaries. First $t0 holds g + h, which is 30. Then $t1 holds i + j, which is 7. Finally $s0, which is f, gets 30 minus 7, which is 23. In every instruction the **first** register is the destination and the next two are sources.

Scene 8.3 — Memory Operands

Visual directions: Memory strip showing byte addresses; a word spanning 4 bytes. Then a Big-Endian vs Little-Endian side-by-side for the word 0x12345678.

Diagram instructions:

```
Word 0x12345678 stored at address 100:
Big Endian (MIPS):    100: 12   101: 34   102: 56   103: 78
Little Endian:        100: 78   101: 56   102: 34   103: 12
```

Narration:

> **Memory operands.** Main memory is used for composite data — arrays, structures and dynamic data. To apply arithmetic operations, we **load** values from memory into registers, and **store** results from registers back to memory. Memory is **byte addressed** — each address identifies an 8-bit byte. Words are **aligned** in memory: a word address must be a multiple of 4. MIPS is **Big Endian**: the most-significant byte is at the lowest address of a word. Compare **Little Endian**, where the least-significant byte is at the lowest address. The data-transfer examples — g = h + A[8] and A[12] = h + A[8] — were worked fully in Topic 1, scenes 1.8 and 1.9.

On-screen revision points:

- Byte addressed; aligned words (address multiple of 4); MIPS = Big Endian.

Scene 8.4 — Constant or Immediate Operands

Narration:

> **Immediate operands** are constant data specified in an instruction, for example `addi $s3, $s3, 4`. There is **no subtract-immediate** instruction — just use a negative constant: `addi $s2, $s1, -1`. Both were worked in Topic 1, scene 1.4.

Scene 8.5 — Arithmetic and Data-Transfer Operations

Visual directions: Two cards.

Narration:

> **Arithmetic operations:** add and subtract use **three operands** — two sources and one destination. Example: `add $t0, $s1, $s2`. **Data-transfer operations** move data between memory and registers: `lw` loads and `sw` stores. In `lw $t0, 32($s3)`, 32 is the **offset** and $s3 is the **base register** — exactly the labels used in the notes.

Scene 8.6 — Logical Operations: Shift, AND, OR, NOR

Visual directions: The notes' table, then 32-bit registers with the highlighted bit columns from the notes' figures.

On-screen table (from notes): | Operation | C | Java | MIPS | |---|---|---|---| | Shift left | << | << | sll | | Shift right | >> | >>> | srl | | Bitwise AND | & | & | and, andi | | Bitwise OR | | | | | or, ori | | Bitwise NOT | ~ | ~ | nor |

Worked examples (values from the notes' figures):

```
$t2 = 0000 0000 0000 0000 0000 1101 1100 0000  (0x00000DC0)
$t1 = 0000 0000 0000 0000 0011 1100 0000 0000  (0x00003C00)

and $t0, $t1, $t2 → 0000 0000 0000 0000 0000 1100 0000 0000  (0x00000C00)
or  $t0, $t1, $t2 → 0000 0000 0000 0000 0011 1101 1100 0000  (0x00003DC0)
NOT $t1 (nor $t0, $t1, $zero)
                  → 1111 1111 1111 1111 1100 0011 1111 1111  (0xFFFFC3FF)

sll $t2, $s0, 4 with $s0 = 9 (0000 1001) → 1001 0000 = 144 = 9 × 2⁴
srl $t2, $s0, 2 with $s0 = 36           → 9           = 36 ÷ 2²
```

Narration:

> Logical instructions manipulate individual bits — useful for extracting and inserting groups of bits in a word. **Shifts** use the R-format, and the **shamt** field says how many positions to shift. **Shift left logical** shifts left and fills with zeros; shifting left by i bits multiplies by 2 to the power i. **Shift right logical** fills with zeros; shifting right by i bits divides by 2 to the power i — unsigned numbers only. (The notes write "2i"; it means 2 to the power i.) So 9 shifted left by 4 is 144, which is 9 times 16. **AND** is useful to **mask** bits: it selects some bits and clears the others to 0. Look at the shaded column: only where both $t1 and $t2 have a 1 do we get a 1. **OR** is used to **include** bits: it sets some bits to 1 and leaves the others unchanged. **NOT** inverts every bit. MIPS has no NOT instruction; it uses **NOR** with $zero, because A NOR 0 equals NOT A.

Scene 8.7 — Conditional Operations and Decision Making

Visual directions: The notes' if-else flowchart: diamond "i == j?", left box "f = g + h", right box "Else: f = g − h", both merging to "Exit".

Narration:

> Based on input data and computed values, different instructions execute. MIPS has two decision-making instructions, similar to an if statement with a go-to. `beq register1, register2, L1` — **branch if equal**: go to L1 if the two registers are equal. `bne register1, register2, L1` — **branch if not equal**: go to L1 if they differ. These are called **conditional branches**. And `j L1` is an **unconditional jump** to L1. (The notes list `j L1` on the same line as `slt`; they are separate instructions.)

Worked example — compiling IF (from the notes):

```
if (i == j) f = g + h; else f = g - h;     f,g,h,i,j in $s0..$s4

      bne $s3, $s4, Else    # if i ≠ j go to Else
      add $s0, $s1, $s2     # f = g + h   (skipped if i ≠ j)
      j   Exit              # skip the else part
Else: sub $s0, $s1, $s2     # f = g − h
Exit: ...

Trace 1: i = 5, j = 5 → bne not taken → f = g + h → j Exit
Trace 2: i = 5, j = 7 → bne taken → Else: f = g − h
(The assembler calculates the addresses of Else and Exit.)
```

Narration (continued):

> Notice the compiler tests the **opposite** condition — bne — so it can skip the then-part. The `j Exit` jumps over the else-part.

Worked example — compiling a loop (from the notes):

```
while (save[i] == k) i += 1;     i in $s3, k in $s5, base of save in $s6

Loop: sll  $t1, $s3, 2      # $t1 = i × 4 (byte offset)
      add  $t1, $t1, $s6    # $t1 = address of save[i]
      lw   $t0, 0($t1)      # $t0 = save[i]
      bne  $t0, $s5, Exit   # exit if save[i] ≠ k
      addi $s3, $s3, 1      # i = i + 1
      j    Loop
Exit: ...

Trace: $s6 = 2000, k = 7, save[0] = 7, save[1] = 7, save[2] = 3, i = 0
 i=0: $t1 = 0 → 2000 → save[0]=7 = k → i = 1
 i=1: $t1 = 4 → 2004 → save[1]=7 = k → i = 2
 i=2: $t1 = 8 → 2008 → save[2]=3 ≠ k → Exit.  Final i = 2
```

Narration (continued):

> sll by 2 multiplies i by 4 to get the byte offset. Adding the base gives the address. lw reads save[i]. bne exits when it differs from k. Otherwise i is incremented and we jump back. This loop uses four addressing modes at once: register (add), immediate (addi), base (lw), PC-relative (bne) — and pseudodirect for j.

Scene 8.8 — More Conditional Operations: slt, slti, Signed vs Unsigned

Worked example (from the notes):

```
slt  rd, rs, rt        if (rs < rt) rd = 1; else rd = 0;
slti rt, rs, constant  if (rs < constant) rt = 1; else rt = 0;

Used with beq/bne:
slt $t0, $s1, $s2      # $t0 = 1 if $s1 < $s2
bne $t0, $zero, L      # branch to L if it was less

Signed vs unsigned:
$s0 = 1111 1111 1111 1111 1111 1111 1111 1111
$s1 = 0000 0000 0000 0000 0000 0000 0000 0001
slt  $t0, $s0, $s1  # signed:   −1 < +1             → $t0 = 1
sltu $t0, $s0, $s1  # unsigned: 4,294,967,295 > +1  → $t0 = 0
```

Narration:

> slt — set on less than — sets the destination to 1 if the first source is less than the second, otherwise 0. slti compares with a constant. Combined with bne and $zero, they make "branch if less than". The same bit pattern means different things signed and unsigned. All ones is −1 as a signed number, so slt says −1 is less than +1, giving 1. As an unsigned number it is about 4.29 billion, so sltu gives 0.

Scene 8.9 — Jump and Link, Jump Register (procedures)

Narration:

> For procedures, the notes give two more instructions. `jal ProcedureLabel` — **jump and link**: the address of the following instruction is put in register **$ra**, the return address, and then it jumps to the target. `jr $ra` — **jump register**: copies $ra into the program counter, returning from the procedure. jr can also be used for computed jumps, for example case/switch statements.

On-screen revision points (Topic 8):

- Instruction set = vocabulary; instruction = word.
- Operands: register (32 × 32-bit), memory (byte addressed, aligned, Big Endian), immediate (no subi).
- Operations: arithmetic, data transfer, logical (sll, srl, and, or, nor), conditional (beq, bne, slt, slti, j, jal, jr).

Scene 9.1 — Topic 9: Instruction Representation

Visual directions: Heading "TOPIC 9 — REPRESENTING INSTRUCTIONS". A line of assembly turns into a row of 0s and 1s.

Narration:

> Instructions are encoded in binary, called **machine code**. MIPS instructions are encoded as **32-bit instruction words**, using a small number of formats that encode the operation code — **opcode** — and register numbers. Register numbers from the notes: $t0 to $t7 are registers 8 to 15; $t8 and $t9 are 24 and 25; $s0 to $s7 are 16 to 23.

Scene 9.2 — R-Format Field by Field

Diagram instructions:

```
| op     | rs     | rt     | rd     | shamt  | funct  |
| 31–26  | 25–21  | 20–16  | 15–11  | 10–6   | 5–0    |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |   total = 32
```

Narration:

> **op** — the operation code, 6 bits. **rs** — the first source register number, 5 bits. **rt** — the second source register number, 5 bits. **rd** — the destination register number, 5 bits. **shamt** — shift amount, 5 bits; zero for non-shift instructions. **funct** — function code, 6 bits, which extends the opcode and selects the variant. 6 + 5 + 5 + 5 + 5 + 6 = 32.

Worked example — `add $t0, $s1, $s2` (notes): 0 | 17 | 18 | 8 | 0 | 32 → `000000 10001 10010 01000 00000 100000` = **0x02324020**.

Worked example — `sub $s0, $t0, $t1`:

```
op=0 000000 | rs=$t0=8 01000 | rt=$t1=9 01001 | rd=$s0=16 10000 | shamt 00000 | funct=34 100010
= 0000 0001 0000 1001 1000 0000 0010 0010 = 0x01098022
```

Worked example — `sll $t2, $s0, 4`: op 0, rs 0 (unused), rt = $s0 = 16, rd = $t2 = 10, shamt = 4, funct = 0.

Scene 9.3 — I-Format Field by Field

Diagram instructions:

```
| op     | rs     | rt     | constant or address |
| 31–26  | 25–21  | 20–16  | 15–0                |
| 6 bits | 5 bits | 5 bits | 16 bits             |   total = 32
```

Narration:

> The I-format is used for **immediate arithmetic** and **load/store** instructions. **rt** is the destination or source register number. The **constant** ranges from −2 to the 15 up to 2 to the 15 minus 1 — that is −32,768 to +32,767. The **address** is an offset added to the base address in **rs**. Example from the notes: `lw $t0, 32($s3)` — temporary register $t0 gets A[8]. 19 for $s3 goes in rs, 8 for $t0 goes in rt, 32 goes in the address field. The notes stress: in a load, rt is the **destination**. Why have more than one format? A compromise: keeping all instructions 32 bits long, while allowing both three-register instructions and instructions with useful 16-bit constants.

Worked examples: `lw $t0,32($s3)` = 0x8E680020; `sw $t0,48($s3)` = 0xAE680030; `addi $s3,$s3,4` = 0x22730004 (all derived in Part 1).

On-screen table (from notes): | Instruction | Format | op | rs | rt | rd | shamt | funct | address | |---|---|---|---|---|---|---|---|---| | add | R | 0 | reg | reg | reg | 0 | 32 | n.a. | | sub | R | 0 | reg | reg | reg | 0 | 34 | n.a. | | addi | I | 8 | reg | reg | n.a. | n.a. | n.a. | constant | | lw | I | 35 | reg | reg | n.a. | n.a. | n.a. | address | | sw | I | 43 | reg | reg | n.a. | n.a. | n.a. | address |

> "reg" means a register number 0–31; "address" means a 16-bit address; "n.a." means the field does not appear in that format. add and sub share op 0; the hardware uses funct to decide: 32 for add, 34 for subtract.

Scene 9.4 — J-Format Field by Field

Diagram instructions:

```
| opcode  | target  |
| 31–26   | 25–0    |
| 6 bits  | 26 bits |   total = 32

PC ← PC[31:28] || IR[25:0] || 00
```

Narration:

> J-type is short for "jump type". It has a 6-bit opcode in bits 31 to 26 and a 26-bit target in bits 25 to 0. The semantics: PC gets the upper 4 bits of the program counter, followed by the 26 target bits — the lower 26 bits of the instruction register — followed by two zeros, creating a 32-bit address. Worked in Topic 1, scene 1.13.

On-screen revision points (Topic 9): | Format | Fields (bits) | Examples | |---|---|---| | R | op 6, rs 5, rt 5, rd 5, shamt 5, funct 6 | add, sub, and, or, slt, sll, srl, jr | | I | op 6, rs 5, rt 5, constant/address 16 | addi, lw, sw, beq, bne, slti, andi, ori | | J | op 6, target 26 | j, jal |

Scene 10.1 — Topic 10: Building a Datapath — The Approach

Visual directions: Heading "TOPIC 10 — BUILDING A DATAPATH". An empty canvas; pieces will be added in five stages, each stage shown with a stage number.

Narration:

> The notes say a reasonable way to start a datapath design is to examine the major components required to execute each class of MIPS instruction. We look at which **datapath elements** each instruction needs, working down through the levels of **abstraction**, and showing the control signals as we go. We already met every element in Topic 3. Now we build the datapath in stages, the way a designer would.

Scene 10.2 — Stage 1: Instruction Fetch and PC + 4

Visual directions: Draw notes' Figure 2.2: PC, instruction memory, adder with 4.

Narration:

> To execute any instruction, we start by fetching it from memory. To prepare for the next instruction, we increment the PC so it points to the next instruction, 4 bytes later. Three elements — PC, instruction memory and an adder — form the fetch datapath.

Scene 10.3 — Stage 2: R-Format — Register File and ALU

Visual directions: Add the register file and ALU (Figure 2.3) to the right of the fetch datapath. Wire instruction fields to the register-number inputs.

Narration:

> R-format instructions read two registers, perform an ALU operation, and write the result. So we add the register file — two read ports, one write port — and the ALU. Instruction bits 25–21 and 20–16 go to the read register inputs; the read data go to the ALU; the ALU result goes back to write data; RegWrite controls the write.

Scene 10.4 — Stage 3: Loads and Stores — Data Memory and Sign Extension

Visual directions: Add data memory and sign-extend unit (Figure 2.4).

Narration:

> For lw and sw we also need to **sign-extend** the 16-bit offset to 32 bits, and a **data memory** to read from or write to. The data memory is written on stores, so it has read and write controls, an address input, and a write-data input. The ALU computes base plus offset as the address.

Scene 10.5 — Stage 4: Branches — Shift Left 2 and Branch Adder

Visual directions: Draw notes' Figure 2.5: "PC+4 from instruction datapath" → Add; Sign-extend → Shift left 2 → Add → "Branch target"; Registers → ALU → Zero → "To branch control logic".

Narration:

> For beq we add a sign-extension path, a **shift left 2**, and a separate **adder** to compute the branch target from PC + 4. The ALU does the comparison and its Zero output goes to the branch control logic, which decides whether PC + 4 or the branch target replaces the PC. Two definitions from the notes: **Branch target address**: the address specified in a branch, which becomes the new PC if the branch is taken — the sum of the offset field and the address of the instruction following the branch. **Delayed branch**: a branch where the instruction immediately following it is always executed, whether or not the condition is true. MIPS uses delayed branches because of pipelining, but our simple design ignores them.

Scene 10.6 — Stage 5: Creating a Single Datapath — Sharing with Multiplexors

Visual directions: The R-type and memory datapaths overlap; the two conflict points glow red; multiplexors drop in to resolve them.

Narration:

> The notes say R-type and memory datapaths are quite similar. The key differences: One — arithmetic-logical instructions use the ALU with **both inputs from registers**; memory instructions use the ALU for the address, with the second input the **sign-extended 16-bit offset**. Fix: a multiplexor — ALUSrc — at the ALU's second input. Two — the value stored into the destination register comes from the **ALU** for R-type or from **memory** for a load. Fix: a multiplexor — MemtoReg — at the write-data input. This simplest datapath executes every instruction in **one clock cycle**. So no datapath resource can be used more than once per instruction, and any element needed more than once must be **duplicated**. That is why we have separate **instruction and data memories**, and separate adders besides the ALU. But many elements can be **shared** by different instruction flows. To share an element between instruction classes, we allow multiple connections to its input, using a **multiplexor** and a **control signal** to select among them. Adding the PCSrc multiplexor to integrate branches completes Figure 4.11 — the complete datapath from Topic 3. Adding the jump path and the RegDst multiplexor (clarifications from Topic 2) completes support for our whole subset.

On-screen revision points (Topic 10):

- Stage order: fetch → R-type (registers + ALU) → load/store (data memory + sign-extend) → branch (shift left 2 + adder) → combine with muxes + control.
- Single cycle → no resource used twice → separate instruction and data memories.
- Sharing → multiplexors + control signals.

FINAL REVISION SECTION

Scene R.1 — Complete Unit 3 Recap

Visual directions: The roadmap from Scene 0.1 returns; every box is now ticked.

Narration:

> We have covered all of Unit 3. Let us revise everything, quickly and clearly. We began with the five MIPS addressing modes, saw how a basic MIPS implementation executes lw, sw, add, sub, and, or, slt, beq and j, and studied each hardware component. Then we learned the eight great ideas, the five functional units, how a stored program is executed step by step, how performance is measured, the MIPS operations and operands, the three instruction formats, and how the datapath is built.

Scene R.2 — Revision: Five Functional Units

On-screen: Input → Memory (primary: RAM, ROM, cache; secondary: disks, tapes, CD-ROM) → ALU → Output; Control unit = nerve centre generating timing signals.

Narration:

> Input accepts coded information. Memory stores programs and data. The ALU performs operations using registers. Output sends results out. Control coordinates everything.

Scene R.3 — Revision: Eight Great Ideas

On-screen: Moore's Law · Abstraction · Common case fast · Parallelism · Pipelining · Prediction · Hierarchy of memories · Dependability via redundancy.

Narration:

> Remember them as a story: plan for **Moore's Law**; tame complexity with **abstraction**; speed up the **common case**; do things in **parallel**; overlap them with **pipelining**; don't wait — **predict**; use a **hierarchy** of memories; and survive failures with **redundancy**.

Scene R.4 — Revision: CPU Performance Equations

On-screen:

```
Performance = 1 / Execution time
Performance_X / Performance_Y = Execution time_Y / Execution time_X = n
CPU time = CPU clock cycles × Clock cycle time = CPU clock cycles / Clock rate
CPU clock cycles = Instruction count × CPI
CPU time = Instruction count × CPI × Clock cycle time
CPU time = User CPU time + System CPU time
```

Scene R.5 — Revision: Instruction Formats

On-screen:

```
R: op6 | rs5 | rt5 | rd5 | shamt5 | funct6   = 32
I: op6 | rs5 | rt5 | constant/address16     = 32
J: op6 | target26                           = 32
Opcodes: R-type 0 (add funct 32, sub funct 34), addi 8, lw 35, sw 43
```

Scene R.6 — Revision: Addressing Modes

On-screen:

```
1 Immediate     operand = 16-bit constant in instruction         addi
2 Register      operand = register                                 add
3 Base          EA = Reg[rs] + sign-ext(offset)                    lw, sw
4 PC-relative   target = PC + 4 + (sign-ext(offset) << 2)          beq, bne
5 Pseudodirect  target = PC[31:28] || address26 || 00              j, jal
```

Scene R.7 — Revision: Instruction Execution Sequence

On-screen (two views):

```
Basic operational concepts:
PC → MAR → Read → MDR → IR → decode → operands (MAR/MDR) → ALU → result (MDR/MAR, write)
→ PC + 1 word

MIPS datapath:
Fetch (PC → instruction memory; PC + 4) → Read registers → ALU → Memory (lw/sw) → Write-
back (R, lw) → PC update
```

Scene R.8 — Revision: Complete Datapath and Component Functions

Visual directions: Full labelled datapath (Scene 3.12) one final time; each component flashes as named.

Narration:

> PC — address of the current instruction. Instruction memory — supplies the instruction. Adder — PC + 4. Register file — two reads, one write. Sign-extend — 16 to 32 bits. ALUSrc mux — register or immediate. ALU — operate, compute address, or compare; Zero flag. Data memory — load and store. MemtoReg mux — ALU or memory to register. Shift left 2 and branch adder — branch target. PCSrc mux — PC + 4 or target, selected by Branch AND Zero. Control unit — decodes the opcode and drives every control line.

Scene R.9 — Final Exam-Oriented Checklist

Visual directions: A checklist with tick boxes, each ticking as it is read.

On-screen checklist:

- ☐ Define ISA, microarchitecture, systems design; compare CISC and RISC.
- ☐ List and explain all eight great ideas with an example each.
- ☐ Draw the five-functional-unit block diagram; define word, word length, address, RAM, ROM, cache, access time; primary vs secondary memory.
- ☐ Explain the 12 operating steps for `Add LOCA, R0` using PC, MAR, MDR, IR.
- ☐ Define interrupt and interrupt service routine.
- ☐ Define response time, throughput, CPU time, user and system CPU time, CPI.
- ☐ Write and apply all performance equations; solve the four problems in the notes.
- ☐ Identify register, memory and immediate operands; Big vs Little Endian.
- ☐ Explain logical ops (sll, srl, and, or, nor) and conditional ops (beq, bne, slt, sltu, j, jal, jr); compile an if-else and a while loop.
- ☐ Draw R, I and J formats with bit widths; encode `add $t0,$s1,$s2` and `lw $t0,32($s3)`.
- ☐ Define and draw all five addressing modes; compute a PC-relative and a pseudodirect target.
- ☐ State the first two steps common to all instructions; explain ALU use per class.
- ☐ Explain the three multiplexors and the control lines (notes' Figure 1.2).
- ☐ Describe each datapath element (PC, memories, register file, ALU, adders, sign-extend, shift left 2, muxes, control).
- ☐ Build the datapath in stages and draw the complete single-cycle datapath.
- ☐ Trace R-type, lw, sw and beq through the datapath.
- ☐ Define branch taken, branch not taken, branch target address, delayed branch.
- ☐ Explain why the single-cycle design is impractical.

Narration:

> Go through this checklist before your exam. If you can do every item — define it, draw it, and work an example — you have mastered Unit 3. Thank you for watching, and all the best for your examination.

End of the Unit 3 video script.